Practical DevOps Engineering

Production delivery patterns from fast feedback to durable recovery

Birol Tilki

Version 1.0 — 15 August 2026

Contents

How to Use This Book

Who this book is for

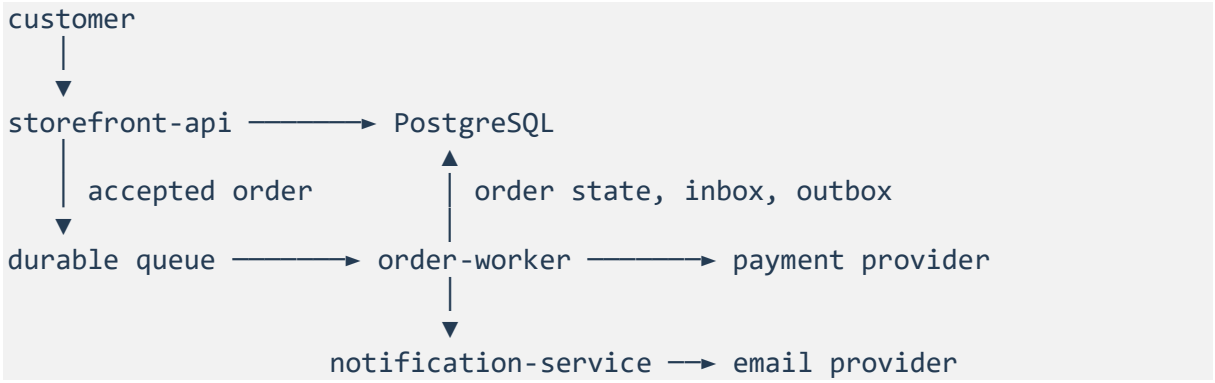
This book is for intermediate-to-advanced DevOps, cloud, infrastructure, platform, and **SRE (Site Reliability Engineering)** practitioners who already work comfortably with Linux, Git, containers, **CI/CD (Continuous Integration and Continuous Delivery)**, cloud infrastructure, Kubernetes fundamentals, infrastructure as code, monitoring, networking, and security fundamentals.

It does not teach those tools from first principles. Each chapter introduces only the conceptual model needed to make a production decision, then guides you through implementing, breaking, diagnosing, and verifying one capability in the cumulative Northwind system.

The book's scope is the production delivery path: from source feedback and artifact identity through deployment, operation, incident recovery, and reconstruction of one environment. Broader software-supply-chain governance belongs to the DevSecOps book. Shared platform products, tenancy, and fleet lifecycle belong to the Platform Engineering book. Portfolio reliability objectives, regional-loss strategy, and recurring recovery programs belong to the SRE book.

The Northwind system

Northwind Commerce is deliberately small enough to understand and large enough to need production controls:



- `storefront-api` serves catalog reads and accepts orders.
- `order-worker` processes accepted orders asynchronously and must tolerate redelivery without duplicating payment or inventory effects.
- `notification-service` handles non-critical confirmation delivery.
- PostgreSQL is the primary stateful component.
- The durable queue decouples acceptance from asynchronous completion.
- Payment and email providers are external dependencies represented by controllable fixtures.

The critical production outcome is:

> A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

Every chapter changes how Northwind protects or proves that outcome.

The cumulative delivery path

The chapters form one dependency chain:

```
fast source feedback
  → verifiable artifact
  → reconciled infrastructure
  → bounded Kubernetes runtime
  → workload identity
  → explainable production behavior
  → progressive release
  → compatible data change
  → safe asynchronous work
  → GitOps reconciliation
  → reliability-constrained cost and capacity
  → coordinated failed-change recovery
  → reconstruction from durable evidence
```

Later chapters consume earlier decisions instead of reteaching them. Artifact digests flow into infrastructure and release policy. Runtime identity becomes dependency authority. Observability becomes rollout evidence. Data compatibility and idempotency become recovery controls. GitOps becomes the normal reconciliation path, while incident and restore chapters define bounded exceptions and return to reviewed intent.

Repository layout

The reader-facing chapters live under:

```
books/devops/
```

The one cumulative implementation lives under:

```
books/labs/devops/northwind/
```

Within the lab:

```
.github/        workflow and review boundaries
services/       Northwind application behavior used by early checks
checkpoints/    chapter capability verifiers
config/         application runtime configuration contracts
data/           schema-evolution contract
delivery/       pipeline and rollout contracts
finops/         cost and capacity decisions
fixtures/       production-shaped observations and failure evidence
gitops/         reconciliation authority and behavior
incident/       failed-change response contract
infra/          infrastructure intent and backend policy
k8s/            runtime manifests
messaging/      asynchronous processing contract
observability/  telemetry and service-level evidence
recovery/       durable restore contract and objectives
release/        artifact trust expectations
security/       workload-identity contract
tools/          behavioral evaluators used by checkpoints
```

Generated reports, build outputs, simulated cloud state, virtual environments, caches, and backup artifacts are intentionally excluded from version control.

How chapter tags work

Every chapter has two tags:

```
chapter-NN-start
chapter-NN-complete
```

These are curated exercise snapshots, not merge milestones. Each start tag contains the cumulative Northwind state needed to begin that chapter and the latest verifier for that exercise. A complete tag is a reference solution and verification target. Because verifier corrections may be applied to both snapshots after a chapter is written, Git ancestry between a start and complete tag is not part of the contract.

Do not merge a complete tag into your working branch and do not infer chapter changes from commit ancestry. Use the tags in one of two ways:

Guided implementation

Create a working branch from the start snapshot:

```
git switch -c my-chapter-NN chapter-NN-start
```

Run the baseline. A successful baseline command means it confirmed the capability is red; it does not mean the production design is already safe.

```
make chapter-NN-baseline
```

Follow the chapter's Practice boxes, run its checkpoint, inject the stated failure, and verify recovery.

Reference comparison

Inspect the complete snapshot without merging it:

```
git diff chapter-NN-start chapter-NN-complete
git show chapter-NN-complete:path/to/file
```

The difference is a teaching aid, not a substitute for following the implementation and interpreting its evidence.

Red, green, and recovery

The lab distinguishes four states:

- **Broken lab:** the verifier cannot run because a required file, dependency, or generated input is unexpectedly missing.
- **Red capability:** the verifier runs successfully and proves the declared production capability is unsafe or incomplete.
- **Green capability:** policy and behavioral evidence satisfy the chapter's independent expectations.
- **Verified recovery:** after the realistic failure, business and system evidence show that the critical outcome is healthy again.

A command completing, a deployment finishing, a rollback executing, a replica count increasing, or a backup restoring is an action. None proves recovery on its own.

Generated state in Chapters 2 and 3

Most completion checkpoints run directly from their complete snapshot. Two chapters intentionally regenerate ignored state.

Chapter 2 creates the local artifact and evidence set before tampering with it:

```
make chapter-02-evidence
make chapter-02-checkpoint
make chapter-02-break
make chapter-02-verify-tamper
```

Chapter 3 recreates the simulated cloud object, imports its state binding, creates a saved plan, and applies it before the completed checkpoint:

```
make chapter-03-reset
make chapter-03-import
make chapter-03-plan
make chapter-03-apply
make chapter-03-checkpoint
```

These generated files are excluded because committed build evidence or simulated infrastructure state would make a fresh exercise appear complete without executing the mechanism.

Best Practice and Production Practice

The book uses two related labels:

- **Best Practice:** a strong default that is broadly useful.
- **Production Practice:** how that default must be validated or adapted for the actual workload, failure modes, security boundaries, cost constraints, dependencies, and organization.

For example, default-deny networking is a strong default. In production, it is useful only when the selected network implementation enforces it, the intended traffic path is explicitly allowed, and operators have proved that policy failure does not make the service silently unreachable.

How to read the Practice boxes

`Theory` establishes the mental model required for the chapter's decisions. It is not a separate academic survey.

`Practice` tells you what to change or prove, why the change matters, which evidence to inspect, and how to verify the result. Guided implementation is the practical work; you are not expected to rediscover essential production steps without help.

`Independent Practice` is the final retrieval exercise. It changes the constraints and requires a justified design rather than copying the guided implementation.

Environment

Use Python 3.12 and Git. Docker, Kubernetes, Terraform, and hosted workflow familiarity are assumed where their production behavior is discussed, but the local deterministic exercises do not require a live cloud account or cluster unless a chapter explicitly says otherwise.

From the Northwind lab root:

```
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make test
make lint
```

The local exercises model production decisions and verify declared behavior. Their disclaimers matter: a simulator that correctly rejects unsafe policy is not evidence that a real cluster, identity provider, billing export, incident channel, database restore, or external dependency has been exercised.

Five recurring principles

The chapters repeatedly use five production principles:

1. **Blast-radius control:** expose the smallest defensible scope to uncertain change or failure.
2. **Explicit contracts:** make identity, state, authority, outcomes, and failure behavior reviewable.
3. **Trustworthy evidence:** separate observations and expectations so a mechanism cannot approve itself.
4. **Reconciliation:** compare desired, recorded, external, and actual state, then make disagreement visible and owned.
5. **Recovery:** distinguish performing a corrective action from proving the critical outcome is healthy again.

The dedicated failure in each chapter names the subset it exercises. By Chapter 13, these principles connect source review to reconstruction of the production environment from durable evidence.

01 — Build Fast Feedback You Can Trust

> **Outcome:** Treat **CI (Continuous Integration)** as a production feedback system, then implement a pipeline whose ordering, critical-path latency, authority, build input, and continued conformance can be verified.

Current Northwind state: `storefront-api` has a health contract and a small test suite. Its pipeline is slow, noisy, and allowed to do too much.
Prerequisites: Python 3.12, Git, and workflow-syntax familiarity; Docker is optional.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. A cheap failure delivered late

Northwind's checkout team made six changes last week. Four were correct on the first attempt. Two failed lint, but engineers waited more than twenty minutes to learn that because the pipeline built a container first.

Signal	Current value
Feedback **p50 (50th percentile)**	**18 minutes**
Feedback **p95 (95th percentile)**	**31 minutes**
Queue time at p95	**10 minutes**
Flake rate	**7%**
Build context	**1.4 gigabytes**
Cheap lint and test work	**165 seconds**

Engineers concluded that hosted runners were too slow. Runner utilization looked ordinary, however, and larger runners would not make lint execute before the container build.

> Why does feedback take 31 minutes when lint and tests finish in under three?
>
> **Practice — Establish the red baseline**
>
> Check out the real unsafe state and prove that its feedback, context, and authority contracts are failing for known reasons.

The implementation repository carries a deliberately unsafe starting state. Enter it and create your working branch:

```
cd books/labs/devops/northwind
git switch -c my-chapter-01 chapter-01-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-01-baseline
```

The command should succeed while reporting that the capability is red. It derives that result from the checked-out workflow, pipeline graph, and Docker context: the budget is exceeded, expensive work precedes cheap checks, generated build input is uncontrolled, and the workflow requests package-write permission. A baseline command should distinguish an expected red state from a broken checkpoint.

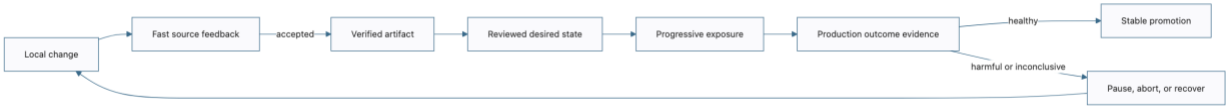
2. The production model: CI is a feedback system

> *Theory — Trustworthy feedback under production constraints*
>
> Build the mental model needed to choose job order, diagnose latency, and constrain workflow authority.

CI does not primarily produce green check marks. It produces evidence about a proposed change. Useful evidence must arrive quickly enough to influence the next action and be reliable enough that engineers do not rerun it reflexively.

```
change
  |
  v
cheap checks --> focused tests --> artifact work --> security evidence
  |                   |                   |
  +------ stop at the first useful failure -------+
```

The developer journey continues beyond this chapter. One change moves through distinct evidence and authority boundaries rather than one all-powerful pipeline:



Chapter 1 owns the first feedback boundary. Chapter 2 establishes artifact identity, Chapter 6 supplies production evidence, Chapter 7 makes that evidence control exposure, and Chapter 10 closes the reviewed reconciliation path. Later chapters add data, asynchronous work, cost, incident, and restoration constraints without replacing this journey.

Four mechanisms matter:

- **Ordering:** cheap, high-signal checks reject bad changes before expensive work.
- **Latency evidence:** queue and execution time are measured separately.
- **Reliability:** a flaky check consumes time without increasing confidence.
- **Authority:** pull-request workflows processing untrusted changes do not receive write or deploy authority. Chapter 2 will add publication through a separately guarded release path.

Measure the path a change actually waits for

A pipeline is a **DAG (Directed Acyclic Graph)**: jobs are nodes and dependency relationships are edges. The elapsed execution time is determined by the longest dependency path, not the sum of all job durations.

For a path containing jobs `j1 ... jn`, Northwind models feedback time as:

```
feedback time = queue time + duration(j1) + ... + duration(jn)
```

The pipeline's critical path is the path with the largest result. Two independent five-minute jobs can complete in roughly five minutes when capacity is available; making them parallel does not help when both wait ten minutes for a runner. Northwind therefore measures queue time separately and optimizes the shortest path that still produces enough trustworthy evidence—not the shortest pipeline at any cost.

The ten-minute p95 budget and 200-megabyte context limit used here are Northwind constraints, not universal targets. Repository checkout, artifact transfer, runner startup, cancellation, and retries can also add latency in a real measurement.

Treat workflow authority as part of execution

A workflow executes repository-controlled code. On a pull request, that code may be supplied by a contributor who should not be able to publish packages, change deployments, or mint privileged credentials. Permissions therefore belong in the pipeline model alongside dependencies and durations.

The safe default is read-only authority with narrower permissions granted only to the job that needs them. Environment approval does not compensate for giving an earlier untrusted job broad credentials: authority must be constrained at the point where code executes.

3. Implement the feedback contract

Establish the application boundary

> **Practice — Verify the behavior CI must protect**
>
> Run the service tests and identify the application contract that later pipeline changes must preserve.

CI needs real behavior to protect. `storefront-api` currently exposes liveness and catalog endpoints; later chapters add orders and stateful dependencies.

```
python -m pytest services/storefront-api/tests -q
```

The tests prove that liveness reports process health, catalog data satisfies its minimal contract, and unknown routes return `404`. Component tests stay with the service. Cross-component tests enter the repository only when those boundaries exist.

Put cheap evidence first

> **Practice — Diagnose the 31-minute critical path**
>
> Calculate the baseline manually, compare it with the analyzer, and then redesign the dependency graph.

Open `delivery/pipeline.json`. Before running the analyzer, use the model above to diagnose the start state. Its only dependency path is:

```
queue → build → lint → test
600 + 1095 + 35 + 130 = 1860 seconds = 31 minutes
```

Now compare your calculation with the executable model:

```
python tools/pipeline_feedback.py \
  --pipeline delivery/pipeline.json \
  --budget-seconds 600 \
  || echo "feedback_budget_exceeded_as_expected"
```

The report should agree with 1,860 seconds. If it does not, inspect the dependencies rather than adjusting the formula to fit the output. This diagnostic separates the 600-second queue delay from the 1,260-second execution path and shows that larger runners alone cannot repair the ordering problem.

The start state puts `build` before `lint`, reproducing the late cheap failure. Edit the dependency graph so that lint must succeed before tests, and tests must succeed before build. The completed graph is:

```
{
  "stages": [
    {"name": "lint", "seconds": 35, "needs": []},
    {"name": "test", "seconds": 130, "needs": ["lint"]},
    {"name": "build", "seconds": 220, "needs": ["test"]}
  ],
  "queue_seconds": 40
}
```

```
python tools/pipeline_feedback.py \
  --pipeline delivery/pipeline.json \
  --budget-seconds 600
```

It reports a 425-second queue-plus-critical path: lint rejects cheap defects, tests establish behavior, and only then does build spend container resources. The analyzer follows the dependency graph rather than adding every job, so independent checks can run concurrently without falsifying the budget.

This linear path fits today's small service. Larger repositories may run independent test groups concurrently after a common cheap gate. Parallelism is useful only when it shortens the critical path without creating excessive queue demand or obscuring evidence.

Make workflow authority explicit

> **Practice — Remove publication authority from pull requests**
>
> Change the workflow to read-only authority and make its dependency order agree with the pipeline model.

The starting workflow grants `packages: write` globally. That permission is unnecessary because Chapter 1 does not publish anything. Edit `.github/workflows/ci.yml` so the workflow default is read-only:

```
permissions:
  contents: read
```

A pull request can modify scripts executed by CI. Giving every job package authority turns pipeline code into a privilege-escalation path. Remove `packages: write` rather than preserving unused authority for a future chapter. `.github/CODEOWNERS` records the review boundary; repository settings must enforce code-owner review and branch protection for it to become a real control.

While editing the workflow, express the same fail-fast graph as the pipeline model: `test` needs `lint`, and `build` needs `test`. The model makes offline reasoning deterministic; the workflow is the executable delivery definition. The checkpoint requires them to agree on the important constraints.

Detect drift from the reviewed workflow contract

Reusable workflows and repository templates reduce duplication, but copying a template once does not keep a service safe. A repository may later remove a required job, bypass an edge, restore write authority, or retain an obsolete template version while still producing valid workflow syntax.

Open `delivery/workflow-conformance.json`. It defines the small interface Northwind expects every service workflow to preserve: template identity, required jobs and dependency edges, evidence-producing commands, read-only default authority, and forbidden write permissions. It deliberately does not require byte-for-byte equality. A service may add workload-specific checks as long as it preserves the reviewed interface.

Add `# northwind-template: ci-v1` to `.github/workflows/ci.yml`, then run the conformance check:

```
make chapter-01-template
make chapter-01-template-drift
```

The first command checks the actual workflow. The second evaluates a syntactically plausible fixture that advertises `ci-v0`, lets tests bypass lint, and restores package-write authority. It succeeds only when that drift is rejected. The contract is owned like production code: changing `ci-v1` requires review and a migration decision for consuming repositories. This is lightweight conformance, not an internal developer platform; Chapter 10 later reconciles reviewed environment intent.

Bound hidden build input

> **Practice — Reject uncontrolled build context**
>
> Observe the generated input crossing the context boundary, exclude it deliberately, and prove the transmitted context is back inside budget.

The baseline creates a sparse `build-output/cache.bin` representing 1.4 gigabytes of generated build output. It consumes almost no physical disk, but its logical size accurately exercises context selection. Run the context gate before changing `.dockerignore`:

```
python tools/context_gate.py --root . --limit-bytes 209715200 \
  || echo "context_rejected_as_expected"
```

Now add `build-output` to `.dockerignore` and run the same gate again:

```
python tools/context_gate.py --root . --limit-bytes 209715200
```

`context_gate.py` reads the repository's `.dockerignore` rather than maintaining a separate exclusion list. The new result must exclude `build-output/cache.bin` and pass before Docker uploads context or consumes a remote builder. The sample patterns are deliberately simple; if your production ignore file uses advanced negation or Docker-specific matching, verify the transmitted context with the actual builder as well.

Prove the capability

> **Practice — Verify the complete feedback contract**
>
> Run one capability checkpoint that checks behavior, ordering, latency, context selection, and workflow authority together.

```
make chapter-01-checkpoint
```

The checkpoint verifies application behavior, dependency ordering, the calculated critical-path budget, Docker-context rules, read-only workflow authority, continued workflow-template conformance, and absence of a floating `latest` tag.

The local fixture does not prove GitHub's real p95. Production must export workflow and queue events, calculate rolling percentiles, and retain the commit and workflow revision associated with every sample.

4. Test the design under failure

Severity: delivery degradation; no production outage.
Potential blast radius: pull-request feedback for one repository.
Bounded by: a feedback-budget gate and publication authority withheld from pull requests.
Primary principles: trustworthy evidence and blast-radius control.

> **Practice — Diagnose and recover from queue regression**
>
> Increase only queue delay, identify which part of feedback time changed, and restore the budget without weakening other controls.

Assume runner demand rises after the pipeline is repaired. In `delivery/pipeline.json`, temporarily change `queue_seconds` from `40` to `400`. Before running anything, calculate the expected result:

```
400 queue + 385 execution = 785 seconds
```

Run the analyzer:

```
python tools/pipeline_feedback.py \
  --pipeline delivery/pipeline.json \
  --budget-seconds 600 \
  || echo "queue_regression_detected_as_expected"
```

The critical path has not changed, so reordering jobs or buying faster build compute addresses the wrong component. The evidence isolates a 360-second queue regression. In production, the repair might be concurrency shaping, runner-pool capacity, or workload scheduling; the fixture represents that recovery by restoring `queue_seconds` to `40`.

```
make chapter-01-checkpoint
```

Recovery is not merely the analyzer returning zero. The checkpoint must still show the intended order, total time inside the teaching budget, clean context, green application tests, and narrow authority. A capacity repair that weakens those controls is not recovery.

5. Production reality

Best Practice: run cheap, high-signal checks early and cache deterministic dependencies.

Production Practice: measure the real critical path. An expensive integration test may carry the evidence that prevents costly failure. Move it later; do not delete it merely to improve a dashboard.

Condition	Likely response
Queue p95 grows while execution is stable	**Reshape runner capacity and concurrency.**
Execution grows while queue remains stable	**Inspect stage work, cache behavior, test selection, and build input.**
Flake rate grows	**Assign ownership, quarantine only with expiry, and repair nondeterminism.**

Cache keys are trust decisions. Untrusted pull requests must not poison caches later consumed by privileged release jobs. Hosted actions and reusable workflows are dependencies too; pin them according to the supply-chain policy developed in Chapter 2.

Production evidence should retain queue and execution duration, result and rerun state, workflow and commit identity, cache outcome, protected-branch context, and estimated runner cost. These measures describe the delivery system; do not turn them into individual-engineer rankings.

6. What changed

Before	After
Expensive work ran before lint.	**Cheap evidence rejects changes first.**
A 1.4-gigabyte context looked like generic slowness.	**Context is measured and rejected before upload.**
Queue and execution delay were conflated.	**The feedback report exposes both.**
Pull-request code had unused package authority.	**Chapter 1 is read-only; Chapter 2 will add a guarded release path.**
A copied workflow could silently drift from its reviewed defaults.	**A versioned conformance contract preserves required evidence, ordering, and authority while allowing service-specific additions.**
Pipeline speed was an opinion.	**A behavioral checkpoint enforces an explicit contract.**

What mattered was not GitHub Actions syntax. Northwind changed CI from a command sequence into a measurable feedback contract with constrained authority.

Durable outputs

Artifact	Location	Keep it because
Reviewed delivery contracts	delivery/pipeline.json and delivery/workflow-conformance.json	They record the feedback budget and the required workflow interface without forcing every repository to be an identical copy.
Pipeline conformance evidence	evidence/chapter-01-green.json	Retain the generated report with workflow and revision identity so a later regression can be compared with the last accepted ordering, authority, context, template, and latency evidence.

What You Learned

Fast feedback is a production contract, not merely a short list of commands. You can now calculate the critical path, separate queue delay from execution time, order evidence by cost and value, constrain pull-request authority, reject uncontrolled build context, detect workflow-template drift, and diagnose a feedback-budget regression without weakening the controls that make the result trustworthy.

Prove It

> **Independent Practice — Preserve evidence under a tighter budget**
>
> Design and justify a production pipeline without copying the guided topology.

Northwind adds a 12-minute integration suite that catches payment-contract regressions missed by unit tests. Running it on every change breaks the ten-minute budget; running it nightly lets incompatible changes merge.

Design a pipeline that preserves fast feedback without discarding the evidence. Specify which checks block every pull request, which work runs concurrently, whether change-aware selection is safe, what blocks merge or promotion, how flakes are owned, and how you will show that lead time improved without increasing change failures.

Next

Northwind can reject bad source changes quickly. It still cannot prove that staging and production receive the artifact CI tested. Chapter 2 establishes immutable artifact identity and promotion without rebuilding.

02 — Build Once and Promote a Verifiable Artifact

> **Outcome:** Build one release candidate, bind evidence to its immutable digest, verify who built it from which source, and carry that same identity into promotion.

Current Northwind state: pull requests receive fast, bounded feedback. The release workflow still builds once for staging and again for production, publishes mutable names, and produces no verifiable supply-chain evidence.
Prerequisites: Chapter 1's completed state, GitHub Actions familiarity, container-image fundamentals, and Python 3.12.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 105–135 minutes.

1. The source revision is the same; the artifact is not

Northwind approves `storefront-api` in staging, then the production job executes the container build again. Both jobs use the same Git revision, but they run at different times against external package repositories and different builder state.

During the last release review, staging recorded this image identity:

```
ghcr.io/northwind-commerce/storefront-api@sha256:7f93...d22a
```

Production recorded:

```
ghcr.io/northwind-commerce/storefront-api@sha256:b451...901e
```

The release ticket says that staging approved version `latest`. That proves only that the tag resolved to something at the time; it does not identify the bytes that were tested. Nobody can answer whether the digest changed because of a legitimate rebuild, an unpinned dependency, changed builder input, or interference after the build.

> How can Northwind prove that the artifact promoted later is exactly the artifact **CI (Continuous Integration)** built and evaluated?
>
> **Practice — Establish the untrusted release baseline**
>
> Check out the two-build release path and prove that artifact identity and supply-chain evidence are absent.

Start from the deliberately unsafe release path:

```
cd books/labs/devops/northwind
git switch -c my-chapter-02 chapter-02-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-02-baseline
```

The baseline command succeeds because red is expected, but its report must show why: two builds, a floating `latest` reference, no **SBOM (Software Bill of Materials)**, no provenance, no attestation, and no digest passed to promotion. This is an observation of the checked-out workflow—not a prearranged answer hidden in the checkpoint.

2. The production model: identity, evidence, and expectations

> *Theory — Artifact identity and chain of custody*
>
> Build the mental model needed to distinguish a source revision, build execution, artifact digest, tag, and trusted evidence.

A tag is a movable name. A digest is an identity derived from content. If one byte changes, the digest changes. Promotion should therefore carry an image reference of the form `name@sha256:digest`, even when human-friendly tags also exist.

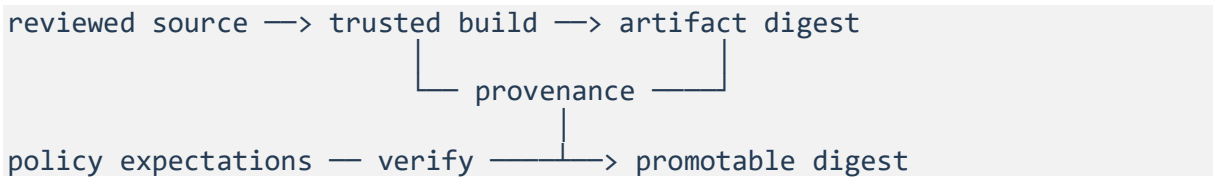
Four identities are often incorrectly treated as one:

Identity	What it identifies	What it does not prove
Source revision	A state in version control	Which dependencies or builder inputs produced the artifact
Build invocation	One execution of a build definition	That its output is the artifact later deployed
Artifact digest	Exact artifact content	Who built it or whether the source was approved
Tag	A registry reference that can move	Stable content identity

The release contract must connect these identities. A reviewed revision enters an expected builder; that invocation produces a digest; authenticated provenance binds the digest to the build facts; policy decides whether those facts are acceptable; promotion carries that digest without another build.

Evidence has different jobs:

- An SBOM describes components associated with an artifact. It helps answer what is inside, but does not by itself prove who produced the artifact.
- **Provenance** binds an artifact subject and digest to build facts such as source, build type, and builder identity.
- An **attestation** is an authenticated statement about a subject. Its value depends on the issuer, signing mechanism, verification policy, and protection of the build system.
- **Expectations** state what Northwind accepts: the artifact name, source repository, revision, builder identity, build type, and evidence type. Validly formatted evidence is not sufficient if it describes an untrusted builder or the wrong source.



The governing promotion invariant is short:

```
build once → identify by digest → verify evidence → promote the same digest
```

3. Define what Northwind is willing to trust

> **Practice — Write independent trust expectations**
>
> Define the artifact, source, revision, builder, and build type Northwind will accept before inspecting generated evidence.

Create `release/expectations.json`:

```
{
  "artifactName": "ghcr.io/northwind-commerce/storefront-api",
  "builderId": "https://github.com/northwind-commerce/storefront/.github/workflows/release.yml@refs/heads/main",
  "buildType": "https://northwind.example/build-types/storefront-container/v1",
  "sourceRepository": "https://github.com/northwind-commerce/storefront",
  "sourceRevision": "chapter-02-complete"
}
```

These are lab expectations, not universal values. In a real verifier, the expected revision comes from the approved release record, the builder identifier must match the identity emitted by the hosted builder, and repository renames or reusable-workflow boundaries require an intentional policy change.

The important ordering is that expectations exist before verification. Deriving the accepted builder or source from the attestation being checked would allow evidence to approve itself.

> **Production callout — AI (Artificial Intelligence)** drafts; humans verify
>
> Treat changes drafted with AI assistance as untrusted source input, not as review evidence. The same protected review, tests, workflow-conformance checks, dependency controls, build isolation, and artifact verification apply regardless of who or what produced the first draft.
>
> Generated Dockerfiles, workflow edits, manifests, and dependency updates can be plausible while changing authority, omitting a failure boundary, inventing a package version, or exposing build data. Keep the proposed diff narrow enough to inspect; do not place production secrets or restricted source into an unapproved model context; run the repository's executable checks; and require an accountable reviewer to explain the operational effect before merge. Record material generation constraints when organizational policy requires traceability.
>
> Provenance later proves which reviewed source and builder produced the artifact. It does not prove that generated source is correct, that a reviewer understood it, or that the model's suggestion was safe. Human approval is therefore not a substitute for verification either: people own the decision, while independent controls produce the evidence.
>
> **Practice — Build one digest-bound evidence set**
>
> Generate the deterministic candidate, manifest, SBOM, and provenance, then identify which claims each file contributes.

Build the local release candidate and its evidence:

```
make chapter-02-evidence
```

The command creates a deterministic teaching artifact at `dist/storefront-api.tar` and writes three evidence files under `evidence/chapter-02/`:

- `manifest.json` carries the immutable artifact reference;
- `sbom.spdx.json` records direct Python dependencies in **SPDX (Software Package Data Exchange)** 2.3 form;
- `provenance.json` binds the digest to the expected source, revision, builder, and build type.

The two direct application dependencies in `requirements.txt` use exact versions, so their declared versions become controlled build inputs and appear in the teaching SBOM. Exact direct pins are not a complete dependency lock: production should resolve and review transitive dependencies, retain the lock artifact, and use hashes or an equivalent integrity mechanism supported by the package workflow. Rebuilding later still remains a separate act from promoting the artifact already evaluated.

Verify the claim, not merely the document

The **SLSA (Supply-chain Levels for Software Artifacts)** build requirements distinguish merely existing provenance from authentic provenance and from a hardened build platform. Apply that distinction to the files you just created. A verifier must answer:

1. Does the subject name and digest match the candidate artifact?
2. Is the statement type understood, and are required fields present?
3. Is the attestation authentic under a trusted issuer or key?
4. Is the builder one Northwind permits?
5. Do source, revision, build type, and external parameters match the approved release?
6. Is the signing identity still acceptable under current policy?

The local verifier exercises subject, format, builder, source, revision, and build-type expectations. It cannot establish authenticity because its local evidence files are unsigned. That limitation is why the hosted workflow later creates an **OIDC (OpenID Connect)**-backed attestation.

> **Practice — Verify subject binding and policy**
>
> Compare the actual artifact digest with its manifest and provenance, then evaluate the evidence against Northwind's independent expectations.

Inspect the binding rather than just checking that files exist:

```
shasum -a 256 dist/storefront-api.tar
python -m json.tool evidence/chapter-02/manifest.json
python -m json.tool evidence/chapter-02/provenance.json
python tools/artifact_evidence.py verify
```

The same digest must appear in the calculated hash, manifest, and provenance subject. The verifier also compares the statement with `release/expectations.json`. This follows the verification shape described by the SLSA artifact verification guidance: validate the subject and digest, then evaluate provenance fields against trusted expectations.

The local evidence generator is intentionally small enough to inspect. It teaches the contract and detects accidental mutation, but an attacker able to replace the artifact could also replace the unsigned evidence files. Authenticity comes from the hosted attestation path implemented next.

4. Build once in the guarded release workflow

Open `.github/workflows/release.yml`. The starting workflow publishes `latest`, then the `promote` job runs `docker/build-push-action` again.

Build once and reproducible builds solve different problems. Build once prevents promotion from changing the candidate. Reproducibility asks whether an independent build of the same declared inputs produces equivalent output and helps expose hidden inputs. It does not justify rebuilding for each environment: dependency availability, timestamps, builder state, or compromise can still change the result. Promotion carries the evaluated digest; reproducibility independently evaluates the build process.

> **Practice — Replace environmental rebuilds with one candidate**
>
> Give one guarded job publication authority, build once, and export its immutable digest for downstream promotion.

Now replace the two-build topology with one build job whose digest becomes an explicit job output:

```yaml
permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
      id-token: write
      attestations: write
    outputs:
      digest: ${{ steps.build.outputs.digest }}
```

The workflow default remains read-only. Only the build job receives package publication and attestation authority. `id-token: write` allows the job to request an OIDC identity token; it does not grant every workflow a long-lived signing key.

Configure the single image build:

```yaml
- name: Build and publish the release candidate once
  id: build
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ghcr.io/northwind-commerce/storefront-api:sha-${{ github.sha }}
    sbom: true
    provenance: mode=max
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

BuildKit attaches SBOM and provenance records to the pushed image. Docker documents the available forms and storage behavior in Build attestations. `mode=max` exposes more build metadata than the default minimal mode; review the resulting statement because build arguments, source locations, and other metadata can disclose information that should never have been passed as secrets.

The `sha-...` tag helps humans search the registry, but it is not the promoted identity. The build step's digest is.

5. Add authenticated evidence, then promote the digest

> **Practice — Attest and promote the build output**
>
> Bind hosted identity to the published digest and pass that same digest into a promotion job that cannot rebuild it.

Immediately after publication, attest the exact build output:

```
- name: Attest the published digest
  uses: actions/attest@v4
  with:
    subject-name: ghcr.io/northwind-commerce/storefront-api
    subject-digest: ${{ steps.build.outputs.digest }}
    push-to-registry: true
```

The subject digest comes from the publishing step; it is not recalculated from a tag. GitHub's current artifact attestation documentation describes the required `id-token`, `attestations`, and package permissions and the container-image subject fields.

Now remove all build and push actions from `promote`. Carry the previous job's output instead:

```
promote:
  needs: build
  runs-on: ubuntu-latest
  environment: production
  permissions:
    contents: read
  steps:
    - name: Record the immutable production candidate
      env:
        CANDIDATE: ghcr.io/northwind-commerce/storefront-
api@${{ needs.build.outputs.digest }}
      run: |
        echo "candidate=${CANDIDATE}" | tee -a "$GITHUB_STEP_SUMMARY"
```

The production environment can add approval and deployment protection rules without receiving build authority. In Chapter 3, this step will submit the verified digest to a reviewed deployment repository. For now it records the handoff and proves that promotion does not rebuild.

> **Practice — Verify the release chain of custody**
>
> Prove that the workflow builds once, produces evidence with scoped authority, and promotes the prior build's digest.

```
make chapter-02-checkpoint
```

Green means more than valid workflow syntax. The checkpoint verifies that the workflow builds once, avoids `latest`, requests scoped authority, enables SBOM and maximum provenance, attests the build output, and passes the prior digest into a non-building promotion job. It also rebuilds the local fixture and verifies every digest and policy expectation.

6. Break the artifact after evidence generation

Severity: blocked release; no production outage.
Potential blast radius: one release candidate.
Bounded by: verification before promotion and an immutable subject digest.
Primary principles: explicit contracts, trustworthy evidence, and recovery.

> **Practice — Detect tampering and rebuild safely**
>
> Mutate the candidate after evidence generation, explain the failed checks, and recover with a new coherent artifact-and-evidence set.

Simulate registry-side mutation or a corrupted transfer by appending bytes after evidence has been generated:

```
make chapter-02-break
make chapter-02-verify-tamper
```

The second command succeeds only when verification rejects the candidate. Its report should show failures for the manifest digest, provenance subject digest, and promotable reference while the expected builder and source remain true. That distinction matters: the evidence has the right shape and claims the right producer, but it no longer describes the available bytes.

Do not repair this by updating the old provenance with the new digest. Evidence belongs to the build execution that produced its subject. Rebuild from the reviewed source and regenerate the evidence:

```
make chapter-02-checkpoint
```

Recovery requires a newly coherent artifact-and-evidence set and a green policy evaluation. A command completing or an image tag existing is not proof of recovery.

7. Production reality

Best Practice: build a release candidate once, refer to it by digest, generate SBOM and provenance during that build, and verify authenticated evidence against explicit expectations before promotion.

Production Practice: secure the system that issues the evidence. A perfect signature from an overprivileged or attacker-controlled workflow faithfully authenticates the wrong process.

SLSA level is not a checkbox

Locally produced provenance demonstrates the structure associated with SLSA Build L1. Authentic provenance from a recognized hosted issuer can support Build L2 when the platform and verification satisfy the specification. Build L3 additionally requires a hardened build platform with isolation properties. Adding `provenance: mode=max` does not by itself make an arbitrary pipeline L3.

Verification policy needs ownership

Define who may change trusted builders, source repositories, reusable release workflows, and admission policy. Protect workflow files with code-owner review and branch rules. Pin third-party

actions according to organizational policy; tags improve maintainability, while immutable commit references reduce movement risk. Record any exception and its expiry.

Multi-platform images have more than one digest

A multi-platform image index has its own digest and refers to platform-specific manifests with their own digests. Decide whether promotion and verification target the index, each platform manifest, or both. Do not compare unlike subjects and conclude that a valid release was altered.

SBOM existence is not SBOM quality

The local fixture lists only direct Python requirements. A production SBOM should account for operating-system packages, language dependencies, generated or vendored components, and the final image filesystem. Completeness, vulnerability policy, exceptions, and remediation belong to the DevSecOps book; this chapter establishes evidence identity and delivery integration.

Registry lifecycle can break recovery

Retain promoted digests and their attached attestations for at least the deployment and rollback horizon. Garbage collection based only on convenient tags can delete an artifact that production still references. Test retrieval and rollback, not just publication.

8. What changed

Before	After
Staging and production rebuilt the same revision.	**One build produces the candidate promoted between environments.**
latest named whichever image it currently referenced.	**The digest identifies the exact promoted content.**
Evidence files were absent.	**SBOM, provenance, and hosted attestation are bound to the build output.**
Valid-looking metadata could define its own trust.	**Independent expectations constrain source, revision, builder, build type, and subject.**
Mutation could remain hidden behind a familiar name.	**Digest verification blocks the changed candidate before promotion.**

Northwind did not merely add signing syntax. It created a chain of custody: reviewed source enters a constrained builder, the builder publishes one immutable subject, evidence describes that subject, policy verifies it, and promotion carries the same identity forward.

Durable outputs

Artifact	Location	Keep it because
Artifact trust expectations	release/expectations.json	They independently define the subject, source, revision, builder, and build type that verification may accept.
Digest-bound release evidence	evidence/chapter-02/manifest.json, evidence/chapter-02/sbom.spdx.json, and evidence/chapter-02/provenance.json	Together they identify the candidate, describe its declared contents, and bind it to build facts for verification and later diagnosis.

What You Learned

Source identity does not prove artifact identity. You can now build one candidate, identify it by digest, bind an SBOM and provenance to that subject, evaluate authenticated evidence against independent trust expectations, promote without rebuilding, detect post-build tampering, and recover by producing a new artifact and evidence set rather than rewriting history.

Prove It

> **Independent Practice — Design a partner artifact contract**
>
> Define the acceptance and recovery policy for a multi-platform artifact built outside Northwind's trust boundary.

Northwind acquires a service built in a partner repository. The partner supplies a signed image, SBOM, and provenance, but uses a builder Northwind does not operate. Production runs both `linux/amd64` and `linux/arm64`, and the registry removes untagged manifests after 30 days.

Design the acceptance and promotion contract. Specify the trusted issuer and builder expectations, source and revision constraints, multi-platform subjects, vulnerability-policy ownership, exception path, registry retention rule, and evidence required before rollback. Explain which controls prevent a compromised partner workflow from approving itself and which risks remain even after cryptographic verification succeeds.

Next

Northwind can now identify and verify the exact artifact entering promotion. It still creates runtime infrastructure through mutable console actions and cannot prove that declared state matches deployed state. Chapter 3 moves infrastructure changes into a reviewed reconciliation path and makes drift observable.

03 — Reconcile Infrastructure Through Reviewed Changes

> **Outcome:** Bring an existing production service under declared ownership, review a saved change plan, separate planning from apply authority, detect drift, and reconcile through the reviewed path.

Current Northwind state: Chapter 2 produces a verified immutable artifact, but production infrastructure is still changed manually and has no trustworthy state binding.
Prerequisites: Chapters 1–2, Terraform workflow familiarity, Git, and Python 3.12.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 105–135 minutes.

1. Production exists, but ownership is implicit

Northwind already runs `storefront-api`. An engineer created it during an incident and later added a workflow that can apply infrastructure changes from a pull request. State remains on whichever machine ran last.

The service is healthy, but three questions have no reliable answer:

- Which resource address owns the remote service?
- Does production match reviewed intent or an emergency console action?
- Can the organization prove that apply executed the plan reviewers saw?

> How can Northwind make reviewed configuration authoritative without destroying the production object it is adopting?
>
> **Practice — Establish the unmanaged baseline**
>
> Check out the real red state and observe which ownership, state, authority, and reconciliation contracts are absent.

```
cd books/labs/devops/northwind
git switch -c my-chapter-03 chapter-03-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-03-baseline
```

The command succeeds because red is expected. Its report should show no desired state, no resource binding, local unlocked state, one role for plan and apply, pull-request apply authority, and no digest-pinned deployment declaration.

The lab uses a deterministic local cloud fixture so every reader can exercise state, locking, stale plans, drift, and recovery without a cloud account. `tools/reconcile_infra.py` models the control semantics; production Terraform commands and backend boundaries are called out alongside it.

2. The production model: three representations, one transition

> *Theory — Desired state, state binding, and observed reality*
>
> Build the model needed to decide whether a difference should be imported, accepted into configuration, or reconciled out of production.

IaC (Infrastructure as Code) does not make a configuration file the truth by declaration. A reconciler reasons across three representations:

```
configuration: what should exist
state:         which declared address owns which remote object
actual:        what the provider reports now
```

Terraform state primarily stores bindings between resource instances and remote objects. It is neither a complete design document nor a safe collaboration mechanism when copied between laptops. HashiCorp recommends remote state for team use and warns against storing state in version control because state can contain sensitive values and requires coordinated writes. Terraform state, remote state.

A plan refreshes its view of actual objects, compares them with configuration and prior bindings, and proposes actions. It is evidence, not a guarantee: provider data or infrastructure may change after planning. A saved plan binds the reviewed actions to the configuration and inputs used to create it, but the apply path must still reject stale observations and serialize writers.

Locking protects state mutation from concurrent writers. It does not prove that a change is correct, prevent console drift, or replace review. Disabling locking because acquisition fails converts an operational problem into possible state corruption. Force-unlock is an exceptional recovery action and should target only a lock known to be abandoned. Terraform state locking.

Drift is a difference, not automatically a defect. An emergency scale-up may be the correct actual state and stale configuration may be wrong. Before reconciliation, classify the difference:

1. revert actual infrastructure to reviewed intent;
2. update configuration to preserve an authorized emergency change;
3. import an existing object into the correct address;
4. stop because the plan exposes an unexpected replacement or ownership conflict.

3. Adopt the existing service without recreating it

> **Practice — Declare intent and import one binding**
>
> Describe the existing service, preserve Chapter 2's artifact digest, and bind the remote object to one declared address.

Create `infra/environments/production/desired.json`:

```
{
  "name": "storefront-api",
  "artifact": "ghcr.io/northwind-commerce/storefront-
api@sha256:ecc55173e6c10ec59cac9538a34ac7142c52e8ff504d7e045ed3103c650efb3
2",
  "replicas": 2,
  "admin_cidr": "10.40.0.0/16"
}
```

The **CIDR (Classless Inter-Domain Routing)** range and replica count are Northwind values, not defaults for other systems. The artifact is the digest verified in Chapter 2; infrastructure does not translate it back into a mutable tag.

Reset the fixture and inspect the unmanaged object:

```
make chapter-03-reset
python tools/reconcile_infra.py status
```

Now import it:

```
make chapter-03-import
python -m json.tool .northwind-infra/state-production.json
```

The state must contain one binding from `northwind_service.storefront` to `svc-northwind-production`. Import establishes ownership; it does not prove the configuration is complete or safe. Terraform likewise expects one remote object to map to one resource address. Declarative `import` blocks allow imports to participate in plan and review, while command-line import changes only state. Import existing resources.

4. Make shared state a protected production dependency

> **Practice — Define the backend and authority contract**
>
> Replace laptop state and one shared administrator role with encrypted remote state, locking, and separate plan/apply identities.

Edit `infra/backend-policy.json`:

```
{
  "backend": "remote",
  "encryption": true,
  "locking": true,
  "plan_role": "infrastructure-planner",
  "apply_role": "infrastructure-applier"
}
```

This policy is the runnable lab contract, not a substitute for configuring a real backend. As rechecked for this release, an Amazon **S3 (Simple Storage Service)** Terraform backend can use `encrypt = true`, bucket versioning, restricted object access, and `use_lockfile = true`; HashiCorp marks DynamoDB-based locking as deprecated and says it will be removed in a future minor version. Backend credentials should come from workload identity or the standard credential chain, not hardcoded configuration. Terraform S3 backend.

State read access is sensitive too. A consumer of selected remote-state outputs generally has technical access to the entire snapshot. Prefer publishing non-sensitive outputs through a narrower configuration store when consumers do not need state access.

5. Reconcile the certificate lifecycle

> **Practice — Provision, renew, and fail closed**
>
> Declare certificate ownership and renewal behavior, verify the replacement served by the endpoint, and prove expiry cannot silently downgrade transport security.

Edit `infra/certificate-lifecycle.json`. Keep the hostname, service owner, issuer, and provisioning method explicit. Reference managed key material without embedding private-key bytes in infrastructure state. For Northwind's provider-neutral lab, use an illustrative 30-day renewal threshold and alert at 21 remaining days when no usable replacement exists. These are policy values, not universal service timings: a managed provider may control when renewal begins and may apply different eligibility rules by certificate origin and deployment. AWS Certificate Manager, for example, automatically renews only eligible certificates and exposes approaching-expiration and renewal-action events whose timing depends on certificate type and account configuration. ACM managed renewal, ACM EventBridge events.

Provisioning is only the first transition:

```
declare hostname and owner
  → issue certificate
  → attach to endpoint
  → request renewal before expiry
  → overlap old and new validity
  → verify the endpoint serves the replacement
  → fail closed if no valid certificate remains
```

The replacement is not proven merely because the issuer created it. Verify the requested hostname, trust chain, validity window, and serial actually served by the production endpoint. Continue serving the old certificate while it remains valid if renewal is delayed; once no valid certificate exists, **TLS (Transport Layer Security)** must become unavailable rather than falling back to plaintext traffic.

Run the lifecycle conformance check:

```
make chapter-03-certificate
```

The evaluator uses `infra/certificate-expectations.json` as independent policy and `fixtures/certificates/renewal.json` as observed timing. It proves that renewal begins before expiry, old and new certificates overlap during the switch, the endpoint serves the verified replacement, an expired certificate fails closed, and plaintext fallback remains forbidden.

These controls cover provisioning, renewal, endpoint conformance, and expiry behavior. Private-key theft, issuer compromise, revocation after compromise, and forensic response belong to the DevSecOps book.

6. Produce reviewable change evidence

> **Practice — Create and inspect a saved plan**
>
> Compare declared and actual state, save the proposed transition, and inspect its configuration and observation identities before apply.

```
make chapter-03-plan
python -m json.tool .northwind-infra/production.plan.json
```

The first plan should contain no changes because the imported object already matches declared intent. It still records:

- the resource address;
- state serial;
- configuration digest;
- observed-infrastructure digest;
- proposed field changes;
- destructive-change count.

In Terraform automation, `terraform plan -out=tfplan` creates a saved opaque plan and `terraform show -json tfplan` produces machine-readable review input. Saved plan files can contain complete configuration, values, and sensitive data, so do not commit them or publish them as unrestricted artifacts. Applying a saved plan executes without another interactive approval; protection belongs before the apply job. Terraform plan, create and apply a saved plan.

```
make chapter-03-apply
```

The local apply rejects a changed configuration digest, changed observed digest, or held lock. It then updates actual infrastructure and state serial under the lock. Those checks model why a reviewed plan must not silently become an unreviewed re-plan.

7. Separate speculative planning from protected apply

> **Practice — Make pull requests read-only**
>
> Replace direct pull-request apply with a read-only plan job and a protected apply job that consumes the saved plan.

Edit `.github/workflows/infrastructure.yml`. The completed authority shape is:

```yaml
permissions:
  contents: read

jobs:
  plan:
    permissions:
      contents: read
    # Create and publish the short-lived saved plan.

  apply:
    if: github.event_name == 'workflow_dispatch'
    needs: plan
    environment: production-infrastructure
    permissions:
      contents: read
    # Download and apply the saved plan; do not use --auto-plan.
```

The repository's complete workflow contains the runnable local commands and short-lived artifact handoff. In production, plan and apply obtain different cloud identities. The planner needs read access sufficient to refresh and propose changes; the applier needs narrowly scoped mutation access. Repository environment protection, identity policy, and state-backend policy must agree—job names alone create no security boundary.

> **Practice — Verify reviewed reconciliation**
>
> Run one checkpoint across desired state, resource binding, certificate lifecycle, backend policy, workflow authority, actual agreement, and immutable artifact identity.

```
make chapter-03-checkpoint
```

Green means the system has an explicit desired state, a single imported binding, a verified certificate lifecycle, a locking/encryption policy, separate roles, protected apply, no pull-request apply path, actual agreement, and a digest-pinned artifact.

8. Detect, classify, and reconcile drift

Severity: capacity and cost deviation; no immediate outage.
Potential blast radius: one production service.
Bounded by: read-only detection, a one-resource plan, saved-plan freshness checks, and state locking.
Primary principles: reconciliation, trustworthy evidence, and recovery.

> **Practice — Reconcile a console scale-up**
>
> Introduce an out-of-band replica change, inspect the proposed correction, decide whether intent or reality should win, and verify recovery.

```
make chapter-03-break
python tools/reconcile_infra.py status
make chapter-03-plan
```

The plan should propose one non-destructive change: replicas from five back to two. Do not apply merely because drift exists. Assume incident command confirms that the scale-up was temporary and the service is stable at two replicas. Reviewed intent should therefore win.

```
make chapter-03-apply
make chapter-03-checkpoint
```

If the five replicas were still required, recovery would instead mean changing `desired.json`, reviewing the cost and capacity effect, producing a fresh plan, and applying that intent. A refresh-only operation can inspect or record drift without changing remote infrastructure; it should not be used to make an emergency change authoritative without review. Manage resource drift.

9. Production reality

Best Practice: keep configuration and state under explicit ownership, plan every change, serialize writes, and apply reviewed intent through constrained automation.

Production Practice: design for provider failure, stale evidence, sensitive state, partial apply, imports, refactors, certificate renewal failure, and emergency actions.

- Partition state by ownership and blast radius, not merely by folder aesthetics. One global state couples unrelated changes and recovery.
- Back up and version state, but exercise restoration. Never edit state files directly; use supported state commands and preserve lineage and serial protections.
- Treat `-target`, `-refresh=false`, forced state push, and force-unlock as exceptional operations with recorded justification and follow-up reconciliation.
- Review replacements and destructive actions separately. A syntactically valid plan can still delete the wrong production object.
- Use `moved` blocks for reviewed address refactors and import blocks for adoption. Never bind one remote object to multiple addresses.
- Define an emergency-change path that records actor, reason, expiry, and follow-up owner. Drift detection without ownership becomes alert noise.
- Monitor certificate issuance and endpoint conformance separately. An issuer can report success while the load balancer continues serving the old serial.

10. What changed

Before	After
Production existed outside declared ownership.	**One state address owns the imported remote object.**
State lived locally without coordination.	**Remote, encrypted, locked state is an explicit policy.**
Pull requests could apply directly.	**Planning is read-only; apply uses separate protected authority.**
Review described source files only.	**A saved plan records the proposed transition and observation.**
Certificate creation implied durable transport security.	**Ownership, renewal lead time, overlap, endpoint verification, alerting, and expiry behavior are explicit.**
Console drift was ambiguous.	**Drift is detected, classified, reconciled, and verified.**
Deployment could lose artifact identity.	**Desired infrastructure carries Chapter 2's immutable digest.**

Northwind now has a reviewed reconciliation path. Configuration expresses intent, state records ownership, planning exposes consequences, apply is serialized and protected, and recovery ends only when actual infrastructure agrees with the chosen intent.

Durable outputs

Artifact	Location	Keep it because
Infrastructure and certificate ownership contract	infra/environments/production/desired.json, infra/backend-policy.json, and infra/certificate-lifecycle.json	Together they record the reviewed resource identity, immutable artifact, desired values, certificate lifecycle, state boundary, locking, encryption, and apply authority.
Plan and drift evidence bundle	.northwind-infra/production.plan.json and evidence/chapter-03-green.json	Retain the reviewed plan with actor, source revision, expiry, and apply result outside the local ignored directory; it is the decision record used to explain adoption, drift, and recovery.

What You Learned

Infrastructure as code is an ownership and reconciliation system, not only configuration syntax. You can now adopt an existing resource without recreation, preserve immutable artifact identity, reconcile certificate issuance and renewal, fail closed after expiry, separate planning from apply authority, require locked remote state, interpret a saved plan, classify drift, and verify recovery through agreement among configuration, state, and observed infrastructure.

Prove It

> **Independent Practice — Adopt a shared production database**
>
> Design an import and state-boundary plan for a database already used by three services without recreating it or granting every team state access.

Specify the resource address and owner, discovery and backup evidence, import method, configuration-completeness check, state boundary, read/apply identities, first saved-plan acceptance criteria, handling of sensitive outputs, rollback limitations, and response to a plan that proposes replacement. Explain how emergency console changes become reviewed intent or are reconciled away.

Next

Northwind can now reconcile the infrastructure that carries its verified artifact. Chapter 4 establishes the Kubernetes runtime contract: scheduling, resource boundaries, health semantics, disruption control, and safe workload identity inside that infrastructure.

04 — Establish a Production Kubernetes Runtime

> **Outcome:** Define when `storefront-api` can start, receive traffic, restart, consume capacity, and tolerate controlled disruption.

Current Northwind state: infrastructure is reconciled and carries the verified image digest, but the workload has no production runtime contract.
Prerequisites: Chapters 1–3 and Kubernetes fundamentals.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. Running is not ready

Northwind deployed one Pod with no resource requests, the default service account, and a liveness probe that calls dependency-sensitive `/health/ready`. During a temporary dependency outage, Kubernetes restarts a healthy process and concentrates traffic on fewer replicas.

> Which signals should withdraw traffic, which should restart a process, and which controls bound the resulting blast radius?
>
> **Practice — Establish the unsafe runtime baseline**
>
> Check out the real manifest and prove that its health, capacity, identity, disruption, and traffic contracts are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-04 chapter-04-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-04-baseline
```

2. The production model: separate control questions

> *Theory — Scheduling, health, and disruption contracts*
>
> Decide which Kubernetes mechanism should react to each kind of failure before configuring it.

Resource requests are scheduling promises; limits bound consumption. Kubernetes schedules from requests rather than recent usage, so illustrative values must be validated with workload evidence. Resource management.

The three probes answer different questions:

Probe	Question	Failed action
Startup	Has initialization completed?	Restart after its startup budget is exhausted.
Readiness	Should this Pod receive traffic now?	Remove it from Service endpoints.
Liveness	Is this process unrecoverably stuck?	Restart the container.

A dependency outage usually belongs in readiness, not liveness. Restarting a healthy process does not repair PostgreSQL and may amplify load. Kubernetes explicitly warns that incorrect liveness probes can cause cascading failure. Probes.

A **PDB (PodDisruptionBudget)** constrains voluntary evictions through the eviction interface; it does not prevent node failure, direct deletion, or Deployment rollout behavior. NetworkPolicy also depends on a network implementation that enforces it. These controls have operational prerequisites, not just valid syntax. Disruptions, networking.

3. Bound scheduling and rollout behavior

> **Practice — Add capacity and rollout contracts**
>
> Replicate the workload, define measured resource envelopes, and bound rollout unavailability.

Edit `k8s/base/runtime.yaml`:

```
replicas: 3
strategy:
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

Add container requests of `100m` processor capacity and `128Mi` memory, with illustrative limits of `500m` and `256Mi`. These are teaching values. Production should compare throttling, working-set memory, out-of-memory termination, latency, and scheduling headroom before adopting them.

Set `terminationGracePeriodSeconds: 30`. Graceful termination still requires the application to stop accepting work, drain requests, and exit within the measured budget.

4. Separate startup, traffic, and restart signals

> **Practice — Implement distinct probe semantics**
>
> Protect startup, withdraw dependency-impaired Pods from traffic, and restart only when process-local health fails.

```
startupProbe:
  httpGet: {path: /health/live, port: http}
  periodSeconds: 2
  failureThreshold: 30
livenessProbe:
  httpGet: {path: /health/live, port: http}
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet: {path: /health/ready, port: http}
  periodSeconds: 5
  failureThreshold: 2
```

The startup budget is 60 seconds. The values express Northwind's current observations, not a universal formula. Readiness can include required dependencies; liveness must remain process-local unless restarting truly repairs the condition.

5. Bound configuration, execution, topology, and disruption

> **Practice — Remove ambient authority and constrain placement**
>
> Declare non-secret configuration, restrict the container, spread replicas, use a dedicated identity, preserve availability during voluntary eviction, and default-deny ingress.

Put non-secret environment and structured runtime values in a ConfigMap, but do not treat ConfigMaps as safe storage for credentials. Chapter 5 establishes runtime credential acquisition and rotation; the DevSecOps book owns broader secret governance, detection, policy, and compromise handling.

Choose configuration interfaces deliberately

Replace the broad `envFrom` import with explicit configuration interfaces. Each mechanism has different observation and change behavior:

Interface	Northwind use	Change behavior
Command flags	Listen address and configuration-file path	Startup contract; change requires a reviewed rollout.
Environment variables	Deployment environment and configuration version	Explicit startup-only scalar values; change requires a rollout.
Read-only mounted file	Structured, non-secret runtime values	May reload only through schema validation and atomic activation.

Mount `/etc/northwind/runtime.yaml` read-only and pass its path explicitly with `--config`. Avoid importing every ConfigMap key into the process environment: an accidental new key should not silently become application behavior. Record `CONFIG_VERSION` explicitly and attach the same version to the Pod so Chapter 6 can correlate behavior with both artifact and configuration identity.

The application configuration contract lives in `config/runtime-contract.json`. It requires a versioned schema, named required keys, type and range checks, unknown-key rejection, and validation before readiness. A bad startup configuration must prevent the new Pod from becoming Ready. A bad dynamic candidate must be rejected atomically while the process retains the last known good configuration and emits the validation result.

Run the invalid-reload exercise:

```
make chapter-04-config
```

The fixture proposes `dependency_timeout_ms: "fast"` and an unknown key. The evaluator must reject `config-v2`, preserve active `config-v1`, keep the already healthy process ready, and emit evidence of the rejected candidate. Keeping readiness is safe here because the active configuration remains verified; partially applying the candidate would make that conclusion false.

This deterministic evaluator proves the declared schema and state transition; it does not run a kubelet projection, filesystem watcher, or application reload loop. Production must test projection delay, watcher loss, rapid successive updates, partial reads, process restart during reload, and telemetry that reports the version actually active in memory.

Chapter 4 defines how the process consumes and validates configuration. Chapter 10 will define how configuration versions are reviewed, promoted, frozen, overridden, and reconciled as desired state.

Run the Pod as non-root with the runtime-default seccomp profile. Disable privilege escalation, use a read-only root filesystem, and drop all Linux capabilities. Provide the required writable path explicitly through an `emptyDir` mounted at `/tmp`; validate actual write requirements rather than copying this path universally.

Add a hostname topology-spread constraint with `maxSkew: 1`. `ScheduleAnyway` expresses a preference without making a small cluster unschedulable. Use stricter scheduling only when enough eligible failure domains exist and pending Pods are an acceptable failure mode.

Add a `storefront-api` ServiceAccount with `automountServiceAccountToken: false`, set `serviceAccountName`, and add a PDB with `minAvailable: 2`. Set the named container port to `8080`, matching the image command, while the Service targets that name.

Add one NetworkPolicy that selects `storefront-api` and denies ingress by omission, then a second policy that allows only gateway Pods from explicitly labelled ingress namespaces to the named `http` port. Default deny without the corresponding allow path would make the application securely unreachable. The complete manifest at `chapter-04-complete` is the reference.

Selectors and named ports are contracts: Deployment, Service, PDB, and NetworkPolicy must select the intended Pods, and the Service and policies must resolve to the port where the image actually listens. Valid configuration with the wrong selector or port protects or routes nothing.

> **Practice — Verify the runtime contract**
>
> Check immutable identity, resources, probes, configuration, restricted execution, topology, rollout, termination, identity, disruption, deny-and-allow policy, selectors, and port alignment together.

```
make chapter-04-checkpoint
```

6. Test dependency failure

Severity: partial traffic withdrawal; restart amplification prevented.
Potential blast radius: one `storefront-api` rollout.
Bounded by: readiness, three replicas, rollout limits, and the disruption budget.
Primary principles: explicit contracts, blast-radius control, and recovery.

> **Practice — Prove dependency failure does not become restart failure**
>
> Simulate a failed required dependency and verify traffic withdrawal while process liveness remains true.

```
make chapter-04-break
```

The report must show both `dependency_failure_withdraws_traffic` and `dependency_failure_does_not_restart` as true. Recovery means readiness returns and endpoints rejoin; a restart counter increasing would show the design is still wrong.

7. Production reality

Best Practice: define resources, distinct probes, narrow identity, graceful termination, disruption tolerance, and network boundaries.

Production Practice: validate every value under real startup, peak load, dependency failure, rollout, node drain, topology loss, and policy enforcement. A PDB can block maintenance; a default-deny policy can isolate the service; a read-only filesystem can expose hidden writes; soft topology constraints do not guarantee separation; and a memory limit can turn normal peaks into repeated termination.

8. What changed

Before	After
Running implied healthy.	**Startup, readiness, and liveness have separate meanings.**
Scheduling had no workload promise.	**Requests and limits define an evidence-tuned envelope.**
One replica absorbed every disruption.	**Replicas, rollout bounds, and PDB constrain availability loss.**
Configuration, writable paths, and execution privilege were implicit.	**Explicit flags, environment keys, a validated mounted file, temporary storage, and restricted security contexts declare them.**
Replicas could concentrate on one node.	**A topology-spread preference reduces correlated placement.**
Default identity and open ingress were implicit.	**Identity, default deny, and a narrow gateway allow path are declared.**
Dependency failure caused restarts.	**Readiness withdraws traffic while liveness preserves healthy processes.**

Durable outputs

Artifact	Location	Keep it because
Workload and configuration runtime contract	k8s/base/runtime.yaml and config/runtime-contract.json	Together they define artifact and configuration identity, interfaces, validation, resources, probe semantics, execution restrictions, placement, disruption, and network behavior.
Runtime conformance evidence	evidence/chapter-04-green.json	Retain the report with manifest revision and cluster-policy version so future workload or platform changes can prove the selectors and safety controls still bind to the intended Pods.

What You Learned

A running Pod is not automatically schedulable, correctly configured, ready for traffic, safe to restart, or resilient to disruption. You can now choose configuration interfaces by change behavior, reject invalid startup and reload candidates, expose configuration identity, define measured resource boundaries, separate startup/readiness/liveness semantics, constrain execution and network access, preserve availability during voluntary eviction, and verify that selectors and ports connect the intended controls to the intended workload.

Prove It

> **Independent Practice — Design the worker runtime contract**
>
> Define probes, resources, termination, disruption, identity, and network behavior for `order-worker`, which consumes at-least-once messages and has no web traffic endpoint.

Justify how readiness applies to a queue consumer, how termination stops fetching and finishes or abandons work safely, which failures merit restart, how duplicate processing is bounded, and what evidence tunes its resource envelope.

Next

The workload now has an enforceable runtime contract. Chapter 5 replaces reusable production credentials with scoped workload identity and a controlled fallback for providers that cannot federate.

05 — Replace Static Secrets with Workload Identity

> **Outcome:** Give `order-worker` attributable, scoped, short-lived dependency access and recover from credential compromise without rebuilding the application.

Current Northwind state: Chapter 4 establishes the runtime-identity pattern for `storefront-api`, but `order-worker` still uses the default service account and implied static dependency credentials.
Prerequisites: Chapters 1–4, Kubernetes service accounts, and authorization-policy fundamentals.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. A service account name is not dependency access

`order-worker` needs the broker, PostgreSQL, cloud services, and the payment provider. The unsafe design mounts long-lived credentials as environment values under the default service account. A copied value remains usable outside the Pod, rotation needs a redeploy, and audit records cannot reliably identify the workload that acted.

> How can Northwind authenticate a workload to each dependency with bounded authority, while still handling a payment provider that accepts only an **API (Application Programming Interface)** key?
>
> **Practice — Establish the reusable-credential baseline**
>
> Check out the broad static-credential design and prove identity, federation, scope, rotation, revocation, break-glass, and evidence controls are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-05 chapter-05-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-05-baseline
```

2. The production model: prove identity, then authorize

> *Theory — Federated workload identity*
>
> Decide which runtime subject may exchange bounded proof for short-lived, dependency-specific access.

A secret answers “who possesses this value?” Workload identity answers “which verified runtime subject is asking, for which audience, through which issuer, and until when?” The dependency or an identity broker validates that proof, applies policy, and returns bounded access.

```
Pod + dedicated service account
              ↓ projected, short-lived proof
trusted issuer → subject + audience + expiry → dependency policy
              ↓
       short-lived scoped access
```

The proof is not authority by itself. The trust binding maps a precise subject—namespace plus service account—to allowed audiences and roles. A valid token for the wrong audience or an untrusted issuer must fail.

Kubernetes recommends short-lived projected service-account tokens; the kubelet rotates them, and recipients should validate audience and expiry. Kubernetes service accounts.

3. Bind a dedicated runtime subject

> **Practice — Replace the default workload identity**
>
> Bind `order-worker` to its dedicated service account and deliver an audience-bound, short-lived, automatically refreshed token through a projected file.

Edit `security/workload-identity.json`. Set `kubernetes_service_account` to `order-worker`. Configure the token as `projected-file`, then enable audience binding, short lifetime, and automatic refresh.

Use a file because rotation replaces projected content while the process remains alive. The client must reread the file or use a credential library that does; reading once at startup converts an automatically rotated token into an eventual outage. Do not mount the projection through `subPath`, because Kubernetes documents that this prevents projected-volume updates from reaching the container. Projected volumes.

Disable automatic service-account token mounting for workloads that do not need it. For `order-worker`, request only the audience used by the identity exchange—not a generic administrative audience.

4. Federate supported dependencies

> **Practice — Exchange proof instead of distributing credentials**
>
> Enable federation for cloud, broker, and database access, then constrain authorization to dependency-specific roles with default deny.

Set the three federation values to true. Set authorization scope to `dependency-specific` and `default_deny` to true.
The exact exchange is provider-specific. A cloud platform may trust Kubernetes **OIDC (OpenID Connect)** claims and issue temporary role credentials; a broker may map the same subject to publish or consume permissions; a database proxy may mint a short-lived login. The common contract is stable:

- trust the exact issuer rather than any cluster,
- match namespace and service-account subject,
- validate the intended audience,
- grant only required actions and resources,
- limit session duration,
- log issuance and authorization decisions.

Amazon's workload-identity documentation illustrates this boundary: **IAM (Identity and Access Management)** permissions can be scoped to one service account, but containers remain outside the identity mechanism's security boundary and node credential paths must still be restricted. IAM roles for service accounts.

5. Handle a provider that cannot federate

> **Practice — Broker the unavoidable payment secret**
>
> Replace the literal environment value with a brokered file reference that supports overlapping rotation and live reload.

Keep `payment_provider.supports_federation` false. Set credential delivery to `brokered-file-reference`, rotation to `overlapping-live-reload`, and enable an overlap window.

Workload identity authenticates `order-worker` to the secret broker; it does not transform the provider's static key into a federated credential. The broker returns or projects the currently authorized provider key without storing that value in Git, an image, a manifest, or an environment variable.

Rotation uses a measured overlap: introduce the new provider key, make workloads reload it, verify successful calls use the new version, then revoke the old key. The overlap is not indefinite. Record ownership, maximum duration, rollback condition, and evidence that no workload still uses the retiring version.

Never log the token, provider key, authorization header, or projected file content. Record safe metadata such as credential version, subject, audience, issuer, expiry, decision, and request correlation.

6. Make revocation and emergency access operational

> **Practice — Define compromise controls**
>
> Allow trust revocation without rebuilding the application and replace shared emergency access with approved, time-bound individual identity.

Enable both revocation controls. Set the break-glass identity to `individual-federated-operator`, then require time bounds and approval.

Short lifetime limits persistence but does not replace revocation. Northwind must be able to disable the subject-to-role binding, quarantine the affected workload, and revoke a brokered provider key independently. Recovery issues fresh access only after workload artifact, configuration, and runtime identity are verified.

Break-glass access is for identity-system or control-plane failure, not convenience. It needs named ownership, strong authentication, narrow emergency scope, expiry, independent approval where available, alerting on use, and a post-use review. A shared administrator token defeats attribution precisely when evidence matters most.

7. Prove identity decisions without exposing credentials

> **Practice — Make the identity contract auditable**
>
> Record subject, audience, issuer, expiry, and authorization decision while requiring credential redaction.

Enable every field under `evidence`, then verify the full contract:

```
make chapter-05-checkpoint
```

The checkpoint proves policy structure, not a live provider exchange. Production must test token refresh during long-running work, issuer-key rotation, clock skew, broker or identity-provider outage, policy propagation delay, node credential isolation, payment-key rollover, and audit-log delivery failure.

8. Reject a stolen token and restore legitimate access

Severity: compromised workload credential with attempted lateral access.
Potential blast radius: `order-worker` and explicitly authorized dependency actions.
Bounded by: exact subject binding, audience validation, short expiry, default deny, dependency-specific policy, and independent provider-key revocation.
Primary principles: explicit contracts, blast-radius control, trustworthy evidence, and recovery.

> **Practice — Exercise credential compromise**
>
> Replay a stolen payment-broker token against an administrative audience after revocation, then prove a verified replacement workload can obtain fresh access.

```
make chapter-05-break
```

The report must show `wrong_audience_is_rejected`, `revoked_binding_is_rejected`, `static_fallback_is_not_exposed`, and `verified_workload_can_recover` as true.

Rejected replay is containment, not recovery. Quarantine the suspect Pod and node when appropriate, revoke its trust binding, rotate any provider secret it could read, search decision logs for the subject and credential version, verify no unauthorized effect occurred, deploy the verified artifact under fresh identity, and confirm legitimate dependency calls recover.

9. Production reality

Best Practice: prefer workload federation, short-lived audience-bound proof, dependency-specific authorization, automatic refresh, independent revocation, and attributable emergency access.

Production Practice: inventory which dependencies really support federation; test token refresh and policy propagation; restrict alternate credential paths; design rotation overlap; protect issuer keys, identity brokers, and audit pipelines; and retain a rehearsed emergency path. A service account, sidecar, or secret store is not safe merely because it exists—its trust and failure boundaries must be verified.

10. What changed

Before	After
The default service account represented multiple workloads.	**order-worker has an attributable runtime subject.**
Reusable values supplied dependency access.	**Supported dependencies exchange short-lived workload proof.**
Wildcard policy made compromise lateral.	**Audience and dependency-specific default-deny policy bound authority.**
Payment credentials were literal environment data.	**An unavoidable provider key is brokered, referenced, reloaded, and rotated.**
Rotation and revocation required redeployment.	**Trust and credential versions change independently of application builds.**
Shared emergency access obscured attribution.	**Individual, approved, expiring break-glass access is audited.**

Durable outputs

Artifact	Location	Keep it because
Workload identity and authorization contract	security/workload-identity.json	It records subject, audience, lifetime, dependency permissions, brokered-secret handling, revocation, and emergency authority without storing credential material.
Identity conformance evidence	evidence/chapter-05-green.json	It preserves the accepted policy checks so future identity or provider changes can be compared with the last verified boundary.

What You Learned

Workload identity separates runtime proof from authorization and removes reusable credentials where dependencies can federate. You can now bind a precise workload subject, constrain audience and policy, handle a non-federating provider honestly, rotate and revoke access independently, and verify compromise recovery without putting credential material into evidence.

Prove It

> **Independent Practice — Survive identity-provider unavailability**
>
> Design `order-worker` behavior when existing credentials expire in 20 minutes but the identity exchange is unavailable for 45 minutes.

Define refresh timing, retry and jitter, cached-credential rules, admission of new work, queue backpressure, payment behavior, fail-open versus fail-closed decisions, alerts, break-glass authority, recovery verification, and evidence that no expired or wrong-audience credential was accepted. Justify how the design avoids turning identity recovery into duplicate order effects.

Next

Northwind now has attributable, bounded dependency access. Chapter 6 makes production behavior explainable while keeping tokens, provider keys, and sensitive payloads out of telemetry.

06 — Make Production Behavior Explainable

> **Outcome:** Correlate user-visible failure with route, dependency, trace, and release evidence, then alert on service-level impact rather than component symptoms.

Current Northwind state: Kubernetes can enforce runtime health, but operators cannot explain an order failure across telemetry signals.
Prerequisites: Chapters 1–5 and monitoring fundamentals.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. Healthy process, failed orders

Payment requests now take more than two seconds and return `503`; catalog reads remain fast and processor utilization is 31%. Northwind's only alert fires above 80% utilization. Text logs have no request or trace identity, and request metrics use `user_id` labels.

> How can an operator prove which user journey, dependency, and release is failing without searching every log or declaring the whole service unhealthy?
>
> **Practice — Establish the telemetry baseline**
>
> Check out the symptom-only state and prove that correlation, cardinality, release identity, user-visible indicators, and burn alerting are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-06 chapter-06-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-06-baseline
```

2. The production model: signals answer different questions

> *Theory — Correlated signals and user-visible evidence*
>
> Choose metrics, logs, traces, and service-level indicators according to the production question each can answer.

Metrics aggregate bounded dimensions efficiently: *which route is failing and how often?* Logs retain event context: *what did this request decide?* Traces connect causal work: *where did time and failure propagate?* Resource attributes identify the emitting service, release, and environment. OpenTelemetry treats metrics, logs, and traces as separate signals connected through context propagation. OpenTelemetry signals, instrumentation.

Do not place `user_id`, `order_id`, or `request_id` in metric labels. Their unbounded value sets multiply time series and operational cost. Preserve those identities in controlled logs or traces where access, retention, and privacy policy can be enforced.

A **SLI (Service-Level Indicator)** measures an outcome such as successful valid order submissions or order latency. A **SLO (Service-Level Objective)** states the acceptable target over a window. Error-budget burn expresses how quickly failures consume the permitted unreliability. Component utilization may help diagnosis, but it is not evidence that customers can place orders.

3. Define the telemetry contract

> **Practice — Structure logs and bound metric dimensions**
>
> Replace free text and user-labelled metrics with correlated event fields and bounded route, status, method, and dependency dimensions.

Edit `observability/contract.json`. Set log format to `json` and include `timestamp`, `level`, `message`, `service`, `route`, `status`, `dependency`, `trace_id`, and `request_id`. Remove `user_id` from request-counter labels.

Route values must be templates such as `/orders/{id}`, not raw paths containing identifiers. Dependency names must come from a controlled set. Structured logging is not permission to emit credentials, payment data, or arbitrary request bodies.

> **Practice — Propagate causality and deployment identity**
>
> Carry trace context across **HTTP (Hypertext Transfer Protocol)**, payment, and queue boundaries and identify which release emitted every signal.

Configure `traceparent` and `tracestate` propagation and require `http.server`, `payment.client`, and `queue.publish` spans. Record `service.name`, `service.version`, and `deployment.environment.name` as resource attributes. The older `deployment.environment` key is deprecated. OpenTelemetry's default propagation uses **W3C (World Wide Web Consortium)** Trace Context, while `service.version` identifies the artifact version associated with telemetry. Context propagation, deployment attributes.

Correlation context crosses trust boundaries. Validate incoming headers, avoid sensitive baggage, and create a new trusted boundary where policy requires it.

4. Measure the order journey

> **Practice — Define user-visible indicators**
>
> Measure valid order success and latency separately from catalog and process health.

Add `order_success_ratio` and `order_latency` to `service_level_indicators`. Define a provisional 99.5% order-success objective over 30 days in `service_level_objectives`; this Northwind value must be validated against user expectations and business risk. Burn rate is meaningless without a target and window. Define eligible events precisely: malformed or unauthorized requests may be excluded when they do not represent service failure; dependency failures after accepting valid work normally count. Record the policy so numerator and denominator cannot drift between dashboard and alert.

Latency needs a distribution, not an average. Choose thresholds and percentiles from user expectations and workload evidence. The fixture names the indicator contract; production instrumentation must emit histogram boundaries suitable for those decisions.

> **Practice — Alert on fast and slow error-budget burn**
>
> Replace processor-only alerting with paired windows that detect urgent and sustained user impact.

Add fast burn (`5m` and `1h`, threshold `14.4`) and slow burn (`30m` and `6h`, threshold `6`) alerts for `order_success_ratio`. These are illustrative Northwind policy values. Validate evaluation intervals, minimum traffic, missing-data behavior, and paging ownership. Multi-window alerts reduce sensitivity to brief noise while preserving response to sustained burn.

5. Verify correlated evidence

> **Practice — Verify the observability contract**
>
> Check log structure, bounded labels, context propagation, release identity, user-visible indicators, and burn alerting together.

```
make chapter-06-checkpoint
```

Green proves the declared contract, not backend ingestion, retention, query correctness, or dashboard usability. Production needs end-to-end telemetry tests from application emission through collection and query.

6. Diagnose payment degradation

Severity: order-submission outage; catalog remains available.
Potential blast radius: the `/orders` journey and payment dependency.
Bounded by: route-specific indicators and causal trace context.
Primary principles: trustworthy evidence, blast-radius control, and recovery.

> **Practice — Isolate user impact without a false global outage**
>
> Evaluate the production-shaped fixture and prove that order failure maps to payment while catalog remains healthy.

```
make chapter-06-break
```

The report must detect order failure, isolate `payment-provider`, and keep catalog healthy. Recovery requires repaired payment behavior plus recovered success and latency indicators over the chosen windows; a quiet processor graph or one successful request is insufficient.

7. Production reality

Best Practice: correlate bounded metrics, structured logs, and traces with stable service and release identity; alert on user-visible outcomes.

Production Practice: control telemetry cost, privacy, sampling, retention, collector failure, clock skew, and missing data. Tail sampling can preserve errors but requires infrastructure; head sampling may discard the trace needed during a rare incident. Metrics should remain useful when tracing is sampled. Telemetry pipelines require capacity limits and monitoring so observability failure does not overload the application it observes.

8. What changed

Before	After
Text events could not be correlated.	**Structured logs carry request and trace context.**
User IDs created unbounded metric series.	**Metrics use bounded operational dimensions.**
Failures lacked causal and release identity.	**Trace propagation and resource attributes connect both.**
Processor utilization drove alerting.	**Order SLIs and burn windows represent user impact.**
Payment failure looked service-wide.	**Evidence isolates orders and preserves catalog health.**

Durable outputs

Artifact	Location	Keep it because
Production telemetry and service-level contract	observability/contract.json	It keeps correlation fields, bounded metric dimensions, release identity, indicator eligibility, objective, and burn-window policy reviewable together.
Observability conformance evidence	evidence/chapter-06-green.json	It records the last accepted contract checks for comparison when instrumentation, alerts, or telemetry backends change.

What You Learned

Production observability must explain user-visible behavior without creating uncontrolled cost or cardinality. You can now correlate logs, metrics, and traces across a request path, bind evidence to a release, define service-level indicators and burn alerts, distinguish an affected journey from a global outage, and recognize when missing or misleading telemetry cannot support a safe operational decision.

Prove It

> **Independent Practice — Design asynchronous order evidence**
>
> Extend correlation and service-level measurement from accepted order through queue redelivery, payment, and terminal state.

Define identifiers for logs and traces, bounded metric labels, producer/consumer spans, duplicate-delivery evidence, success and latency eligibility, sampling behavior, and an alert that detects permanently stuck orders without paging on normal queue delay.

Next

Northwind can now explain production behavior. Chapter 7 uses those service-level signals to advance, pause, or abort a progressive release.

07 — Release Progressively and Abort Safely

> **Outcome:** Expose a bounded production cohort to an immutable candidate, advance only on sufficient user-impact evidence, and prove abort restored the prior stable release.

Current Northwind state: Chapter 6 explains production behavior, but deployment still exposes every user before evaluating that evidence.
Prerequisites: Chapters 1–6 and basic rollout-controller familiarity.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. Ready candidate, failed orders

Northwind deploys `storefront-api:latest` to 100% of traffic. The new Pods are Ready, but order success falls to 88% and **p95 (95th percentile)** latency rises to 2.3 seconds because payment calls fail. Stable was serving 99.8% successfully.

> How can Northwind limit exposure, make progression an evidence-based decision, and prove rollback restored the exact prior release?
>
> **Practice — Establish the all-at-once baseline**
>
> Check out the unsafe rollout and prove that cohort, evidence, authority, abort, and rollback contracts are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-07 chapter-07-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-07-baseline
```

2. The production model: rollout as feedback control

> *Theory — Bounded exposure and evidence-driven progression*
>
> Decide when a release controller should advance, pause, or abort without confusing runtime readiness with user success.

A rolling update controls replacement rate. A canary controls exposure and evaluates the candidate against a stable control. Kubernetes availability proves Pods satisfy runtime contracts; it does not prove customers can complete orders. Google defines canarying as a partial, time-limited production deployment whose evaluation determines whether rollout proceeds. Canarying releases.

```
immutable candidate
        ↓
bounded cohort → enough evidence? ── no ─→ pause
        │ yes
        ↓
success and latency pass? ── no ─→ abort → stable digest → verify recovery
        │ yes
        ↓
advance exposure
```

The controller needs three outcomes. **Advance** means evidence is sufficient and acceptable. **Pause** means evidence is inconclusive—often low traffic or missing telemetry. **Abort** means evidence is sufficient and unacceptable. Treating missing data as success converts telemetry failure into uncontrolled progression.

3. Preserve stable and candidate identity

> **Practice — Pin both sides of the comparison**
>
> Replace mutable tags with exact stable and candidate digests and make rollback target the same stable identity.

Edit `delivery/rollout.json`. Set `stable_artifact` and `candidate_artifact` to `name@sha256:digest` references. Set `rollback.target` to exactly the stable reference.

The fixture's repeated a candidate digest is visibly illustrative. In production, the candidate digest must come from Chapter 2's verified release path. A human-friendly tag may coexist, but comparison, promotion, and rollback use digests. Otherwise “rollback” can resolve to different bytes from those previously observed.

Close the handoff explicitly in the same contract:

```
"handoff": {
  "candidate_source": "verified-release-digest",
  "proposal_path": "reviewed-delivery-change",
  "promotion_path": "controller-authored-reviewed-change",
  "stable_transition": "copy-candidate-after-100-percent-and-recovery-
verification"
}
```

Chapter 2 produces the verified digest. A reviewed delivery change writes that digest to `candidate_artifact`; the rollout controller may observe it but cannot invent another artifact. After the 100% stage and final recovery verification pass, protected automation proposes a reviewed change copying the candidate digest to `stable_artifact`. An abort leaves stable unchanged. This closes the source-to-artifact-to-infrastructure chain introduced in Chapters 2 and 3; Chapter 10 will automate reconciliation of the reviewed delivery repository rather than redefine ownership.

4. Bound exposure and require evidence

> **Practice — Define progressive cohorts**
>
> Replace 100% initial exposure with monotonic 5%, 25%, 50%, and 100% stages separated by observation pauses.

```
"steps": [
  {"traffic_percent": 5, "pause_seconds": 300},
  {"traffic_percent": 25, "pause_seconds": 600},
  {"traffic_percent": 50, "pause_seconds": 900},
  {"traffic_percent": 100, "pause_seconds": 0}
]
```

These cohort sizes and pauses are Northwind teaching values. Low traffic may require longer windows or synthetic transactions; high-risk changes may require smaller cohorts. Cohort assignment must avoid bias—for example, routing only internal users to canary can hide production behavior.

> **Practice — Gate progression on volume and outcomes**
>
> Require enough candidate requests, order success and latency limits, automatic abort, and pause on inconclusive evidence.

Configure at least 50 candidate requests, minimum success `0.995`, maximum latency p95 `750` milliseconds, and both `order_success_ratio` and `order_latency` indicators. Set `automatic_abort: true` and `inconclusive_action: pause`.

A **SLI (Service-Level Indicator)** threshold in rollout policy is not automatically the long-window **SLO (Service-Level Objective)** from Chapter 6. The release gate may be stricter and shorter, but its eligibility rules must remain compatible. Minimum sample volume prevents a candidate from advancing after one lucky request; it does not make a small sample statistically representative.

5. Protect promotion and verify the contract

> **Practice — Separate rollout observation from promotion authority**
>
> Require the protected production release identity to advance traffic and require post-rollback indicator verification.

Set `promotion_authority` to `production-release-approver` and `rollback.verify_indicators` to true. In a real controller, repository environment protection, deployment identity, traffic-router authority, and audit events must enforce these names.

> **Practice — Verify the controller contract**
>
> Check immutable identities, reviewed candidate/stable handoffs, bounded monotonic cohorts, evidence sufficiency, user-impact gates, pause/abort behavior, protected promotion, and rollback recovery together.

```
make chapter-07-checkpoint
```

The local contract models controller decisions. Production must also test routing accuracy, telemetry freshness, controller availability, concurrent rollouts, and what happens when abort cannot update the traffic provider.

6. Abort a Ready but harmful candidate

Severity: candidate order-submission failure; stable remains healthy.
Potential blast radius: the configured 5% canary cohort.
Bounded by: immutable identities, limited traffic, minimum evidence, and automatic abort.
Primary principles: blast-radius control, trustworthy evidence, explicit contracts, and recovery.

> **Practice — Prove automatic abort and stable recovery**
>
> Evaluate the failing payment candidate and verify that readiness cannot override user-impact evidence.

```
make chapter-07-break
```

The fixture supplies 50 candidate requests, 88% order success, 2.3-second latency, and `ready: true`. The result must show `ready_is_not_success`, `candidate_aborts`, and `stable_remains_healthy` as true.

Abort is an action, not recovery. Production recovery requires traffic returned to the stable digest, order indicators back within policy, no continuing candidate traffic, and retained candidate-specific traces and logs. Do not erase the evidence by immediately relabelling the failed digest.

7. Production reality

Best Practice: canary immutable artifacts, expose a bounded cohort, require sufficient user-impact evidence, pause uncertainty, and automate abort.
Production Practice: account for low traffic, cohort bias, delayed failures, telemetry gaps, stateful compatibility, traffic-router failure, overlapping releases, and rollback that cannot undo external side effects. Kubernetes Deployment progress reports replica availability and stalled rollout mechanics; business-result analysis requires an additional controller or release system. Kubernetes Deployments.

8. What changed

Before	After
Every user received the candidate immediately.	**Exposure advances through bounded cohorts.**
Tags identified stable and rollback targets.	**Digests preserve exact stable and candidate identity.**
Digest handoff and stable promotion were implicit.	**Reviewed changes carry verified candidates in and promote stable only after full verification.**
Readiness implied release success.	**Order success, latency, and volume control progression.**

Before	After
Missing evidence had no defined action.	**Inconclusive analysis pauses.**
Rollback was a mutable action.	**Abort restores the stable digest and requires recovery evidence.**

Durable outputs

Artifact	Location	Keep it because
Progressive-delivery decision contract	delivery/rollout.json	It binds stable and candidate digests to cohort steps, evidence volume, advance/pause/abort decisions, promotion authority, and recovery verification.
Rollout conformance evidence	evidence/chapter-07-green.json	It preserves the verified controller policy that future threshold, routing, or authority changes must intentionally replace.

What You Learned

Progressive delivery is a feedback controller, not merely a slower Deployment. You can now preserve stable and candidate identity across a reviewed handoff, bound initial exposure, separate insufficient evidence from failed evidence, gate advancement on user-visible outcomes, and distinguish an abort action from verified service recovery.

Prove It

> **Independent Practice — Design a low-traffic release policy**
>
> Adapt the rollout for a service that receives only 20 valid orders per hour but has high financial impact.

Specify cohort assignment, minimum evidence, maximum wait, synthetic evidence, pause and human-review rules, success and latency gates, rollback identity, delayed-failure monitoring, and the conditions under which the release may never advance automatically.

Next

Northwind can now control stateless release risk. Chapter 8 changes persistent data while old and new application versions coexist.

08 — Change Data Without Breaking Compatibility

> **Outcome:** Change an order schema while stable and candidate versions coexist, migrate existing rows safely, and preserve application rollback.

Current Northwind state: Chapter 7 limits candidate traffic, but stable and candidate replicas can still access the same persistent data during rollout and abort.
Prerequisites: Chapters 1–7, relational database transactions, and basic migration tooling.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. A safe binary rollout meets an unsafe schema

Northwind wants to rename `orders.total_cents` to `orders.order_total_cents`. The first migration adds the replacement as required, the candidate writes only the replacement, and the same release drops the legacy column. During the 5% canary, stable replicas still read and write the legacy field. An abort restores the old binary—but not the column it needs.

> How can Northwind evolve persistent data while old and new versions overlap, background migration is incomplete, and application rollback remains possible?
>
> **Practice — Establish the incompatible baseline**
>
> Check out the unsafe schema change and prove that coexistence, migration, validation, enforcement, cleanup, and rollback contracts are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-08 chapter-08-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-08-baseline
```

2. The production model: expand, migrate, contract

> *Theory — Compatibility across independently timed phases*
>
> Decide which schema and application changes may run concurrently without making deployment or rollback depend on one perfectly timed event.

A schema change and an application rollout are separate distributed transitions. Connections remain open, replicas update at different times, a backfill may take hours, and rollback may reintroduce the old binary. A production-safe change therefore preserves compatibility across three phases:

```
expand                  migrate                          contract
Add compatible shape → Move reads/writes &data → Remove legacy shape later
old + new run           retries are safe                 old is absent
```

Expand adds an optional structure without invalidating existing readers or writers. **Migrate** changes application behavior and existing data while both representations remain valid. **Contract** enforces the final invariant and removes the legacy structure only after evidence proves no supported version depends on it.

This pattern buys rollback time at the cost of temporary schema and application complexity. It is not a command sequence to run without observation: each transition has an entry condition, evidence, and a separate recovery decision.

3. Expand without invalidating stable

> **Practice — Add a compatible representation**
>
> Replace the direct rename with a nullable additive field and explicitly preserve the old writer during version coexistence.

Edit `data/schema-change.json`. Set `strategy` to `expand-migrate-contract`, keep `total_cents`, add `order_total_cents` as nullable, and set `coexistence.old_writer_supported` to true.

Adding a nullable column is the model, not a universal claim that every database executes the change without blocking. Before production, inspect the database engine and version, table size, lock mode, rewrite behavior, replication lag, and transaction timeout. PostgreSQL, for example, documents that `ALTER TABLE` subcommands require different lock levels and that some forms rewrite the table and indexes. A logically compatible statement can still cause an operational outage. PostgreSQL `ALTER TABLE`.

4. Move behavior while both versions run

> **Practice — Make writes and reads coexist**
>
> Dual-write both fields and let the candidate read the new value with a legacy fallback.

Set `writes.mode` to `dual-write` and `coexistence.new_reader_fallback` to true. During the overlap:

```
write: total_cents = value, order_total_cents = value
read:  order_total_cents if present, otherwise total_cents
```

The application should update both values in one database transaction. Dual writing through two independent requests creates a new partial-failure problem. If multiple services write the table, inventory every writer—including jobs, administrative tools, and old consumers—before declaring coexistence safe.

The fallback is intentionally temporary. Instrument how often it is used. A falling count proves migration progress; a persistent count reveals an undiscovered writer or failed backfill.

5. Backfill as resumable production work

> **Practice — Define a bounded, retry-safe backfill**
>
> Require batches, idempotent updates, durable progress checkpoints, and validation of both row coverage and values.

Set `backfill.batched`, `backfill.idempotent`, and `backfill.checkpointed` to true. Set validation to `counts-and-values`.

A safe worker updates only missing replacements, commits bounded batches, records its cursor, and can repeat a batch without changing a correct row. Batch size is a measured control, not a constant: tune it against lock duration, replica lag, transaction logs, foreground latency, and available capacity. Throttle or pause when those signals cross Northwind's tested limits.

Counting non-null rows detects missing coverage but not incorrect values. Compare values or invariants as well—for this change, every migrated row must satisfy `order_total_cents = total_cents`. Sample-based validation may supplement, but not silently replace, a complete invariant when the consequence is financial corruption.

6. Delay enforcement and destruction

> **Practice — Put evidence before contraction**
>
> Require retirement of the old version and validated backfill before enforcement, then move destructive cleanup to a later reviewed release.

Set both enforcement prerequisites to true. Set `contract.drop_legacy_column` to `later-reviewed-release` and `rollback_preserves_legacy_reads` to true.

For a legacy cutover, also complete `cutover`: shift traffic progressively, test old and new versions together, and define abort as restoring the old route while keeping the expanded schema. Require zero observed legacy writes, validated backfill, and closure of the agreed rollback window before exiting dual-run. These are separate facts: a complete backfill does not prove that an undiscovered legacy writer has stopped.

“Candidate reached 100%” is not enough. Confirm no old replica, worker, scheduled job, rollback target, or externally maintained client still uses the legacy field. Then enforce the new invariant. Drop the old column only after the agreed rollback window closes and telemetry shows zero legacy reads and writes.

Application rollback and schema rollback are not symmetric. Rolling back application traffic is often fast; reversing a destructive data migration may require restoring data. Northwind therefore rolls back the binary while keeping the expanded schema, then repairs forward unless evidence requires a separately rehearsed data recovery.

Treat dual-run as a temporary transition with an owner and deadline, not a permanent compatibility layer. Record which path is authoritative, compare old and new outcomes, and stop the cutover when correctness, latency, or capacity evidence crosses its abort boundary. Retire the legacy path only through a reviewed decision backed by the exit evidence; do not infer retirement from low traffic alone.

> **Practice — Verify the compatibility contract**
>
> Check expansion, coexistence, backfill safety, validation gates, delayed enforcement, separate cleanup, and application rollback together.

```
make chapter-08-checkpoint
```

The checkpoint verifies the declared migration protocol. Production must additionally rehearse engine-specific locking, cancellation, replication behavior, backup restoration, connection-pool pressure, and the effect of a partially completed backfill.

7. Keep an old writer safe during canary

Severity: order-write incompatibility during a mixed-version release.
Potential blast radius: orders handled by stable replicas while the candidate and backfill coexist.
Bounded by: additive schema, transactional dual writes, reader fallback, delayed enforcement, and separate destructive cleanup.
Primary principles: explicit contracts, blast-radius control, trustworthy evidence, and recovery.

> **Practice — Exercise mixed-version compatibility**
>
> Let an old stable replica write only the legacy field, then prove the candidate can read the row and application rollback preserves the data.

```
make chapter-08-break
```

The scenario must report `old_writer_is_accepted`, `new_reader_handles_legacy_row`, `rollback_preserves_data`, and `legacy_write_blocks_dual_run_exit` as true. A passing check does not mean migration is complete; it proves the deliberately mixed state remains serviceable and correctly prevents premature retirement.
Recovery is not merely stopping the candidate. Return traffic to the verified stable digest, leave the compatible expansion in place, stop or pause the backfill safely, verify new orders remain correct, and retain progress so the migration can resume without duplicating effects.

8. Production reality

Best Practice: use expand–migrate–contract, keep transitions backward compatible, make backfills bounded and idempotent, validate before enforcement, and separate destructive cleanup.
Production Practice: test the exact database engine and dataset; account for locks, replicas, long transactions, caches, change-data-capture consumers, old jobs, and rollback windows. Some changes require shadow tables, online schema-change tooling, or application-level translation rather than a direct alteration. The safe mechanism follows observed workload and failure behavior.

9. What changed

Before	After
One release renamed and removed a live field.	**Expansion, migration, enforcement, and cleanup are independently gated.**
Stable and candidate required incompatible schemas.	**Old writers and new readers coexist.**
The backfill was an unbounded one-time action.	**Batches are retry-safe, checkpointed, throttleable, and validated.**
Candidate completion authorized destruction.	**Old-version retirement and migration evidence authorize later contraction.**
Application abort depended on restoring removed data.	**Rollback keeps the compatible expanded schema and preserves legacy reads.**
Legacy retirement followed assumed completion.	**Measured legacy writes, validated data, and the rollback window gate dual-run exit.**

Durable outputs

- Schema evolution and cutover contract: `data/schema-change.json`.
- Mixed-version failure evidence: `fixtures/data/old-writer-during-canary.json` and the Chapter 8 checkpoint report.

What You Learned

Persistent data makes release phases overlap even when deployment tooling appears sequential. You can now design a compatibility window, move reads and writes without a flag day, run a resumable backfill, distinguish validation from completion, abort a legacy cutover without reversing compatible data, and retire the old path only after its exit evidence is complete.

Prove It

> **Independent Practice — Evolve an order state safely**
>
> Replace a free-text `orders.status` field with a constrained state model while an external reporting consumer updates only once per day.

Define the expansion, old/new read and write behavior, invalid-value handling, backfill cursor and throttle signals, validation queries, consumer evidence, enforcement gate, rollback window, cleanup authority, and response to a backfill stopped at 63%. Justify which step is reversible and which requires restore evidence.

Next

Northwind can now change synchronous application data safely. Chapter 9 applies the same compatibility and evidence discipline to at-least-once asynchronous work, where retries can duplicate external effects.

09 — Operate Asynchronous Work Safely

> **Outcome:** Process at-least-once order messages without duplicating payment or inventory effects, then diagnose and recover from redelivery safely.

Current Northwind state: Chapter 8 preserves database compatibility, but `order-worker` still treats each broker delivery as new work.
Prerequisites: Chapters 1–8, database transactions, queue consumers, and external service calls.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 100–130 minutes.

1. Payment succeeded; acknowledgement disappeared

`order-worker` reserves inventory and the payment provider accepts the charge. The worker commits the order state, but its acknowledgement is lost before the broker records it. The broker delivers the same order again. If the worker equates delivery with intent, the retry can charge the customer twice and reserve inventory twice.

> How can Northwind make redelivery safe when a database, broker, and external payment provider cannot share one atomic transaction?
>
> **Practice — Establish the unsafe consumer baseline**
>
> Check out the consumer that has no durable identity, transaction boundary, bounded retry policy, quarantine path, or outcome evidence.

```
cd books/labs/devops/northwind
git switch -c my-chapter-09 chapter-09-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-09-baseline
```

One control is already correct: delivery is explicitly at least once. The baseline is red because acknowledging that fact without engineering for duplicates is not a safe capability.

2. The production model: repeated delivery, one business effect

> *Theory — Idempotent effect boundaries*
>
> Decide which identity and durable records make repeated execution converge on one observable business outcome.

A broker delivery is an attempt, not a unique business intent. At-least-once systems may redeliver after consumer failure, acknowledgement loss, lease expiry, or broker recovery. Amazon **SQS (Simple Queue Service)** documents that standard queues can deliver a message more than once and advises idempotent consumers. Amazon SQS at-least-once delivery.

```
stable operation identity
          ↓
receive → recognize or claim → perform idempotent external effect
          ↓                            ↓
     commit inbox + order state + outbox
          ↓
     acknowledge delivery
```

“Exactly once” is not a property Northwind assumes across the whole path. The broker, database, and payment provider have different commit points. Northwind instead makes each effect retry-safe, stores durable evidence of the transition, and acknowledges only after the required outcome is recoverable.

3. Give the operation a stable identity

> **Practice — Preserve identity across attempts**
>
> Replace delivery-generated identity with one stable business-operation identifier and serialize conflicting transitions per order.

Edit `messaging/order-worker.json`. Set `message_identity` to `stable-business-operation-id` and `ordering_scope` to `per-order`.

The identifier represents the intent—such as the first payment attempt for order 1042—not the broker's receipt handle or retry count. Every redelivery carries the same identifier. A later, intentionally distinct payment attempt needs a new identifier; otherwise idempotency can suppress legitimate work.

Per-order ordering does not require globally serial processing. Northwind may partition by order, lock the order row, or use a conditional state transition. The necessary guarantee is narrower: two workers cannot concurrently advance the same order through incompatible states.

4. Make local effects atomic

> **Practice — Build a transactional inbox and outbox**
>
> Uniquely record consumed operations with the order transition and stage outgoing events in the same database transaction.

Set the deduplication store to `transactional-inbox`; require a unique identity and atomic state transition. Enable `transactional_outbox` and set `publication.order_event` to `same-transaction-outbox`.

The inbox unique constraint turns a race into a database decision. The order update and inbox record commit together, so a redelivery can distinguish completed work from a new operation. The outbox closes the opposite gap: the order state and the intent to publish its event commit together, while a separate relay retries broker publication.

The relay can still publish the same outbox row more than once if it crashes after send and before marking it sent. Downstream consumers therefore remain idempotent. The outbox removes lost publication between database commit and broker send; it does not remove duplicates.

5. Bound the external payment effect

> **Practice — Reuse identity at the payment boundary**
>
> Send the message identity as the provider idempotency key for every retry of the same payment operation.

Set `processing.payment_idempotency_key` to `message-id`.

The idempotency key identifies the payment operation; it is not the payment credential. Chapter 5's workload identity authenticates `order-worker` to the secret broker, which supplies the referenced provider credential without placing it in this message or contract. Keep authentication material out of queue payloads, inbox rows, outbox events, and retry evidence.

The database cannot atomically commit with an external provider. If the provider accepts idempotency keys, send the same key and identical operation parameters on every retry. Stripe, for example, documents that repeated requests with the same idempotency key return the saved result instead of performing the operation twice. Provider retention windows and error semantics vary, so Northwind must test the actual contract and retain its own durable operation record. Stripe idempotent requests.

A safe sequence is:

1. Receive the stable operation identifier.
2. Return the stored result if the inbox already contains a terminal outcome.
3. Call payment with that identifier as its idempotency key.
4. In one database transaction, record the inbox outcome, transition the order, and append the outbox event.
5. Acknowledge only after commit.

If the worker crashes after payment but before database commit, redelivery calls payment with the same key, obtains the prior result, and completes the local transaction. This is why database deduplication alone is insufficient for an external financial effect.

6. Control leases, acknowledgements, and retries

> **Practice — Make retry timing an explicit contract**
>
> Acknowledge after durable outcome, cover normal processing with a renewable lease, and retry only transient failures with bounded exponential backoff and jitter.

Set acknowledgement timing to `after-durable-outcome`. Require a lease longer than the processing timeout with renewal support. Set retry classification to `transient-only`, backoff to `exponential`, enable jitter, and choose a finite maximum such as the illustrative value `8`.

The lease must exceed expected processing time but cannot replace idempotency: a paused process or network partition can outlive it. Renewal needs an upper bound so a stuck worker cannot hide a message forever.

Retry connection timeouts, rate limits, and explicitly retryable provider failures. Do not repeatedly retry invalid payloads or forbidden transitions. Exponential backoff reduces pressure; jitter prevents many workers from retrying in lockstep. Attempt count, backoff, and lease duration are workload-specific values that production evidence must tune.

7. Quarantine poison messages and measure progress

> **Practice — Make terminal failure recoverable**
>
> Quarantine exhausted or non-retryable messages, preserve their identity, require reviewed replay, and measure duplicates, oldest backlog age, and terminal outcomes.

Enable quarantine, identity preservation, and reviewed replay. Enable all three evidence fields.

Complete `legacy_cutover` for a transition from the legacy worker. Keep the legacy worker as the initial external-effect owner and run the new worker in `shadow-without-external-effects` mode. Both paths may parse, validate, and derive an expected terminal outcome; they must not both charge payment or reserve inventory. Transfer ownership with one reviewed switch, and preserve the original operation identity across both routes so abort does not turn old work into new financial intent.

A dead-letter queue is isolation, not resolution. Its runbook must retain payload version, operation identity, source queue, failure classification, attempt history, and relevant trace context. Replay must use the original identity; minting a new identifier converts replay into a new payment operation. Repair the cause, select the exact messages, bound replay rate, and watch both source and destination outcomes.

Queue depth alone is ambiguous because normal traffic changes it. Oldest-message age shows whether work is making timely progress. Duplicate count reveals retry pressure, while terminal order outcomes distinguish a busy worker from a useful one.

Use `fixtures/messaging/dual-run-cutover.json` as the observed cutover evidence. The new path shadows 10,000 operations without external effects, terminal outcomes reconcile with no unexplained mismatch, the legacy backlog reaches zero, identities remain stable, and the rollback window closes. The checkpoint calculates whether those observations permit retirement; setting an `enabled` flag cannot bypass the decision. If any condition fails, keep or restore the legacy owner, preserve the inbox and provider identities, and diagnose before shifting ownership again.

> **Practice — Verify the asynchronous processing contract**
>
> Check identity, concurrency, inbox/outbox atomicity, payment idempotency, acknowledgement, lease, retry, quarantine, replay, and outcome evidence together.

```
make chapter-09-checkpoint
```

The model verifies the declared protocol. Production must also test broker outage, provider timeout after acceptance, database failover during commit, lease-renewal failure, relay duplication, message expiry, and replay under load.

8. Redeliver after a successful payment

Severity: duplicate financial and inventory effects after acknowledgement loss.
Potential blast radius: one order per unsafe duplicate; a retry storm can widen this across the active backlog.
Bounded by: stable operation identity, provider idempotency, transactional inbox/outbox, per-order serialization, and bounded retries.
Primary principles: explicit contracts, trustworthy evidence, reconciliation, blast-radius control, and recovery.

> **Practice — Prove acknowledgement loss is safe**
>
> Redeliver an operation whose payment and database state succeeded, then verify the second attempt converges without another charge.

```
make chapter-09-break
```

The scenario must report `redelivery_is_detected`, `completed_inbox_skips_effect`, `second_charge_is_suppressed`, `committed_outcome_survives_ack_loss`, and `redelivery_can_be_acknowledged` as true. The completed inbox prevents a second call in this scenario. Reusing the provider key remains a separate defense for a crash after provider acceptance but before the inbox transaction commits.

Acknowledging the duplicate is an action, not complete recovery. Verify exactly one provider-side payment for the operation key, one valid inventory transition, one correct terminal order state, no stuck outbox record, and recovered oldest-message age. Retain the duplicate and acknowledgement-loss evidence for diagnosis.

9. Production reality

Best Practice: assume redelivery, give each intent stable identity, make local state atomic, make external effects idempotent, acknowledge after durable outcome, classify and bound retries, and quarantine poison messages.

Production Practice: validate provider idempotency retention, broker lease and ordering semantics, database contention, partition distribution, outbox relay lag, replay authorization, privacy retention, and capacity during dependency recovery. Ordering can reduce concurrency and create hot partitions; choose the smallest ordering scope that protects the business invariant.

10. What changed

Before	After
Every delivery represented new work.	**A stable operation identity survives redelivery.**
Database and broker writes could diverge.	**Inbox, order state, and outbox define recoverable transaction boundaries.**
A payment retry could repeat the charge.	**The provider receives the same idempotency key for the same intent.**
Acknowledgement and retry timing were implicit.	**Durable outcome, leases, classifications, backoff, and limits govern retries.**
Failed messages disappeared into repeated attempts.	**Quarantine and reviewed replay preserve identity and evidence.**
Queue depth implied worker health.	**Age, duplicates, and terminal outcomes measure useful progress.**
Old and new workers could both own side effects during migration.	**Shadow comparison precedes one reviewed ownership switch, and reconciled evidence gates legacy retirement.**

Durable outputs

- Asynchronous effect and legacy-cutover contract: `messaging/order-worker.json`.
- Executable dual-run evidence: `fixtures/messaging/dual-run-cutover.json` and the Chapter 9 checkpoint report.

What You Learned

At-least-once delivery moves correctness into the consumer and its effect boundaries. You can now distinguish an attempt from an intent, combine inbox and outbox records with business state, extend idempotency to an external provider, design bounded retry and quarantine paths, and migrate worker ownership without allowing dual-run to duplicate external effects.

Prove It

> **Independent Practice — Recover a payment-provider outage**
>
> Design the response when payment times out for 40 minutes, 60,000 orders queue, and the provider recovers with half Northwind's normal request capacity.

Define operation identity, retry classification and schedule, lease behavior, backlog admission, per-order ordering, worker concurrency, provider rate limiting, oldest-age alerting, quarantine criteria, recovery ramp, reconciliation queries, and evidence that no order was charged twice or permanently lost. Justify which controls prevent recovery from becoming a second outage.

Next

Northwind can now process asynchronous work safely. Chapter 10 makes the reviewed delivery repository continuously reconcile environments without giving every automation path production authority.

10 — Reconcile Environments with GitOps

> **Outcome:** Continuously reconcile reviewed production intent through a bounded pull controller, then stop and recover when validly reviewed intent is operationally harmful.

Current Northwind state: Chapters 2 and 7 define verified digest handoffs, but release automation and operators still need an enforceable path from reviewed intent to actual environment state.
Prerequisites: Chapters 1–9, declarative Kubernetes configuration, and repository protection fundamentals.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 100–130 minutes.

1. A reviewed repository is not yet reconciliation

Northwind's release job can identify the verified candidate digest, but it writes directly to the cluster. The same identity can propose, apply, and effectively approve production changes. Manual drift remains until someone notices it, while plaintext secrets can enter the declared state.

> How can Northwind make the reviewed repository authoritative without turning either release automation or the reconciliation controller into an unrestricted production principal?
>
> **Practice — Establish the push-based baseline**
>
> Check out the direct-write design and prove source, handoff, controller authority, reconciliation, exception, and recovery contracts are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-10 chapter-10-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-10-baseline
```

2. The production model: declared intent plus continuous feedback

> *Theory — Pull-based reconciliation authority*
>
> Decide which system may propose, approve, observe, and apply a production transition without allowing any one mechanism to approve itself.

GitOps is not “configuration stored in Git.” The OpenGitOps principles require desired state to be declarative, versioned and immutable, pulled automatically, and continuously reconciled. OpenGitOps principles.

```
verified artifact → proposed repository change → protected review
                                                        ↓
actual cluster ← bounded reconciler pulls ← versioned desired state
       │                                                ↑
       └──── health + drift evidence ──────────────────┘
```

Repository history proves what intent was accepted. The controller observes actual state and attempts convergence. Neither mechanism proves the intent is safe: review can approve a harmful value, health can be misleading, and a powerful controller can amplify error quickly. Production GitOps therefore combines separation of authority, bounded application, health feedback, and an explicit recovery path.

3. Make production intent reviewable and safe to store

> **Practice — Protect source and promotion handoff**
>
> Require safe production intent, then separate candidate proposal, review, and evidence-backed stable promotion.

Edit `gitops/reconciliation.json`. Set the source to declarative and versioned, protect the production path, require `verified-digest`, and set secrets to `references-only`.

The repository stores the Chapter 2 digest and Chapter 7 rollout contract, not a mutable image tag. It stores references understood by Chapter 5's credential delivery path, not payment keys, database passwords, or projected tokens. Repository encryption can reduce accidental disclosure but does not make broad decryption authority safe; identity and audit boundaries still apply.

Protect changes by path and environment. Require review from owners who understand the affected production boundary, prevent self-approval, retain the evaluated revision, and rerun policy when the branch changes. A review accepted against an older revision is not evidence for newly pushed content.

Application configuration follows the same authority path, but it has a different identity from the binary. Edit `gitops/config-delivery.json`: reference Chapter 4's runtime contract, require protected review, promote configuration through development, staging, and production, and keep `artifact_and_config_separate` true. A timeout change should not require rebuilding the image, and deploying a new image must not silently change the accepted configuration. Record both identities in deployment evidence.

Set application of a candidate to `validate-then-atomic`, retain the last-known-good configuration when runtime evidence fails, and report the active configuration version. Schema validation proves shape and type; it does not prove production behavior. The promotion gate must therefore combine Chapter 4 validation with Chapter 6 rollout evidence.

4. Close the candidate and stable handoff

Set `release_automation` to `propose-change`, `merge_authority` to `protected-review`, and the candidate field to `candidate_artifact`. Set the stable transition to `reviewed-after-rollout-verification`.

This completes the contract promised in Chapters 2 and 7:

1. The build produces and verifies a digest.
2. Release automation proposes that digest as the candidate; it cannot merge or reach the cluster.
3. The reconciler observes the merged candidate intent and supplies it to the rollout controller.
4. The rollout controller advances, pauses, or aborts using Chapter 7 evidence.
5. After 100% exposure and recovery verification, protected automation proposes copying the candidate digest to stable.
6. Review accepts or rejects that transition; the reconciler applies accepted intent.

An abort leaves stable unchanged and retains the failed candidate revision. Do not silently edit Git history or relabel a digest to make the repository appear green.

5. Bound the reconciliation controller

> **Practice — Give the controller only the authority it needs**
>
> Switch to pull reconciliation, make repository access read-only, scope cluster writes to Northwind's namespace, and use Chapter 5 workload identity.

Set controller mode to `pull`, repository access to `read`, cluster scope to `northwind-namespace`, workload identity to true, and `can_write_source` to false.

Pull removes production cluster credentials from **CI (Continuous Integration)** and release jobs. It does not justify cluster-admin for the controller. Separate controllers or service accounts by environment and trust boundary; restrict resource kinds, namespaces, and privileged fields; and prevent one tenant's desired state from creating identities that escape its boundary.

The controller can report status outside the desired-state repository through deployment events or an evidence channel. It cannot merge a revert, change approval rules, or rewrite the declaration it consumes. Otherwise a compromised reconciler can manufacture both intent and evidence.

6. Classify drift, pruning, and dependency order

> **Practice — Define continuous but bounded convergence**
>
> Detect drift continuously, self-heal only classified safe differences, guard deletion, order dependencies, and stop unhealthy synchronization after a finite wait.

Enable continuous reconciliation and drift detection. Set self-heal to `safe-drift-only`, pruning to `allowlisted-with-confirmation`, dependency ordering to true, use the illustrative 900-second health timeout, and set failed health action to `pause-and-alert`.

Not every difference should be overwritten automatically. Replica drift inside an owned Deployment may be safe to reconcile; an emergency scale-up, changed persistent-volume claim, or unknown cluster resource needs classification. Chapter 3's rule still holds: drift is a difference before it is a defect.

Pruning is more dangerous than applying an additive field. Allowlist resources the controller owns, preview deletions, protect stateful objects, and define orphan handling. Dependency order prevents an application from racing its namespace, identity binding, policy, migration, or custom-resource definition. Argo **CD (Continuous Delivery)**, for example, applies sync waves in order and waits for earlier out-of-sync or unhealthy waves before progressing. Argo CD sync waves.

A timeout stops indefinite automation; it does not prove rollback. Values depend on startup, migration, and controller behavior and must be tuned with observed evidence.

7. Make emergency divergence temporary

> **Practice — Bound exceptions and run the reconciler trace**
>
> Require temporary, audited divergence, then execute proposal, denial, pull, pause, reviewed revert, and recovery transitions.

Enable all exception controls.

An operator may need to suspend reconciliation before an emergency change. Record who suspended it, why, affected scope, current and expected state, expiry, and recovery owner. Apply the same rule to an emergency application-configuration override: bind it to an individual identity, cap its lifetime, audit it, and automatically remove it at expiry. Before expiry, either encode the emergency state in a reviewed change or deliberately reconcile it away. A forgotten suspension or override creates unmanaged state while dashboards still advertise GitOps.

Run the deterministic local reconciler before the acceptance checkpoint:

```
make chapter-10-trace
make chapter-10-config
make chapter-10-checkpoint
```

The trace executes state transitions rather than accepting enum values as behavior. It attempts a direct cluster write from release automation, proposes and merges a harmful candidate, pulls it, reaches the health timeout, pauses while preserving the source revision and stable artifact, denies a controller source rewrite, merges a distinct reviewed recovery revision, and reconciles back to healthy state. Inspect the ordered events and final state; the checkpoint independently requires those transitions.

The configuration trace proposes reviewed `config-v2`, promotes it through development and staging, and then observes a 0.91 production order-success ratio against the illustrative 0.995 gate. It must freeze the candidate, keep `config-v1` active, leave the artifact digest unchanged, and recover through a reviewed configuration revert. It also advances beyond an emergency override's expiry and proves that the override is removed and represented by a backport revision. These transitions distinguish review, validation, runtime acceptance, active state, and recovery instead of treating a changed file as successful configuration delivery.

This remains a deterministic local control-plane simulator, not a running Git server, Kubernetes cluster, or Argo CD installation. It proves the authority and state-machine contract without requiring external infrastructure. Production must additionally test repository outage, controller restart, webhook loss, polling delay, identity expiry, policy-engine failure, partial sync, cluster unreachability, deletion protection, and multiple controllers claiming the same object.

8. Stop faithfully applying harmful intent

Severity: reviewed candidate degrades order processing while reconciliation repeatedly attempts convergence.
Potential blast radius: the candidate cohort and resources owned by the Northwind production controller.
Bounded by: immutable candidate identity, progressive traffic, health timeout, paused retry, namespace scope, protected stable intent, and reviewed recovery.
Primary principles: reconciliation, trustworthy evidence, explicit contracts, blast-radius control, and recovery.

> **Practice — Exercise a harmful reviewed revision**
>
> Apply a validly reviewed candidate whose runtime and order evidence degrades, then prove the controller pauses without rewriting source or disturbing stable traffic.

```
make chapter-10-break
```

The report must show `review_does_not_equal_safe`, `controller_stops_amplification`, `stable_traffic_is_preserved`, `controller_does_not_rewrite_intent`, and `recovery_requires_health_evidence` as true. These results come from the executed trace: the failure fixture supplies observations, while the state machine decides whether each actor may transition source or cluster state.

Pause is containment, not recovery. Retain the failed revision, digest, controller events, and rollout evidence. Create a reviewed revert removing the candidate or restoring the last accepted declaration. Reconciliation applies the new revision; then verify stable digest, runtime health, order success and latency, absence of continuing candidate traffic, and no orphaned resources.

Optional Game Day — Contain a synchronization storm

> **Game Day — Reconcile without overwhelming the control plane**
>
> Start three controllers against the same reviewed revision, degrade the control plane, and prove global coordination bounds application, unhealthy dependencies stop later waves, pruning is suspended, retries are staggered, and source remains unchanged.

Run the deterministic exercise:

```
make chapter-10-game-day
```

The fixture gives each of three controllers demand for eight concurrent applies, while the shared contract permits ten and the degraded control plane can tolerate twelve. The simulator must therefore report a peak of ten—not eight per controller and not twenty-four in aggregate. It applies the six-resource foundation wave, then the twenty-four-resource workload wave. Unhealthy workload evidence prevents the optional twenty-resource wave, seven requested prunes become zero, and the reviewed source revision remains `7f21storm`.

These values are illustrative. In production, derive concurrency from measured control-plane latency and throttling, controller count, object size, admission cost, and recovery demand. Inject the event in a non-production environment first; define abort conditions, observers, evidence retention, and restoration ownership before starting. A green local trace proves the decision model, not a running controller or Kubernetes control plane.

9. Production reality

Best Practice: keep desired state declarative and versioned, pull it through a continuously reconciling controller, separate proposal from approval, use immutable artifacts and secret references, bound controller authority, and recover through reviewed intent.

Production Practice: classify self-heal and prune behavior per resource, test repository and controller failure, model propagation time, coordinate concurrency across controllers, isolate environments, constrain custom resources, protect stateful deletion, validate health semantics, control emergency suspension, and retain a non-Git recovery path for loss of the repository or controller. Chapter 13 will exercise restoration of those control-plane dependencies.

10. What changed

Before	After
Release automation wrote directly to production.	**It proposes a verified digest for protected review.**
The controller could change its own source.	**It reads desired state and reports evidence through a separate path.**
Tags and plaintext secrets entered declarations.	**Digests and identity-backed secret references are required.**
Drift and deletion behavior were implicit.	**Detection, classified self-heal, guarded pruning, and ownership are explicit.**
Synchronization retried unhealthy state forever.	**Health timeout pauses amplification and retains stable traffic.**
Emergency suspension could become permanent.	**Exceptions expire, remain audited, and require backport or reconciliation.**
Binary and application configuration changed as one implicit release.	**Independently identified configuration is reviewed, promoted, reconciled, and recovered against the Chapter 4 runtime contract.**

Durable outputs

- Reconciliation authority contract: `gitops/reconciliation.json`.
- Application configuration delivery contract and executable promotion trace: `gitops/config-delivery.json` and `tools/config_delivery.py`.

What You Learned

GitOps makes reconciliation continuous; it does not make declared intent correct. You can now connect verified artifact and application-configuration promotion to a protected repository, bound a pull controller with workload identity, classify drift and pruning, stop unhealthy convergence, manage emergency divergence, and recover through a reviewed revision without allowing automation to approve itself.

Prove It

> **Independent Practice — Reconcile two production clusters safely**
>
> Extend Northwind to a second cluster that must trail the first by at least 30 minutes and may run a different secret-provider integration.

Define repository layout, promotion order, digest identity, controller subjects and scopes, health evidence, minimum soak, environment-specific references, drift ownership, partial-promotion recovery, emergency suspension, and the reviewed condition that advances or reverts the second cluster. Explain how correlated bad intent is bounded when both controllers trust the same repository.

Next

Northwind now continuously reconciles reviewed production intent. Chapter 11 measures and controls the capacity and cost consequences of that delivery system without trading away reliability evidence.

11 — Control Delivery Cost and Capacity

> **Outcome:** Reduce cost per successful order through measured capacity changes that preserve latency, backlog, dependency, and recovery contracts.

Current Northwind state: Chapter 10 reconciles production intent, but resource and delivery costs are not connected to service ownership, useful outcomes, or recovery capacity.
Prerequisites: Chapters 1–10, resource requests and limits, autoscaling fundamentals, and service-level evidence.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. A cheaper system that cannot finish orders

Northwind halves `order-worker` capacity. Monthly compute cost falls 34%, and cost per successful order appears 8% lower. Order acceptance still succeeds 99.8% of the time, so the change looks efficient. Yet **p95 (95th percentile)** completion reaches 13 minutes, the oldest message is 31 minutes old, and the backlog does not drain.

> How can Northwind optimize spend without treating delayed work, reduced recovery capacity, or shifted shared cost as savings?
>
> **Practice — Establish the ungoverned baseline**
>
> Check out the cost-only design and prove allocation, unit economics, capacity, forecasting, optimization, and governance contracts are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-11 chapter-11-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-11-baseline
```

2. The production model: cost per useful outcome

> *Theory — Unit economics constrained by reliability*
>
> Decide whether a cost change improves the economics of a successful production outcome rather than merely reducing a billing total.

Total spend answers an accounting question, not an engineering outcome. Northwind needs three connected views:

```
allocated spend        useful outcomes          operating constraints
owner/service/env  ÷  successful orders   +   latency, backlog, recovery
         └──────────── cost per useful outcome ──────────────┘
```

Allocation makes cost attributable. Unit economics normalizes it against a business outcome. Reliability and capacity constraints prevent the denominator from hiding degraded service. If failed or permanently delayed orders disappear from the denominator, unit cost can improve because the system did less useful work.

The **FinOps (Financial Operations)** Framework treats allocation, forecasting, budgeting, anomaly management, and unit economics as connected practices rather than a one-time cost-cutting exercise. FinOps Framework.

3. Make spend attributable

> **Practice — Define attributable unit economics**
>
> Allocate direct and shared spend, then normalize it by a quality-gated successful order while measuring delivery cost separately.

Edit `finops/capacity-contract.json`. Enable the four allocation fields and set shared-cost policy to `documented-allocation`.

Use provider billing dimensions, Kubernetes labels, account or project boundaries, and workload metadata that survive into cost exports. Validate coverage and cardinality; a label standard that only 70% of spend follows does not produce trustworthy service economics.

Shared cost is not free. Allocate cluster control, observability, networking, artifact storage, and shared runners through a declared method such as measured consumption, reserved capacity, or an agreed proportional rule. Show both direct and allocated values so a changed formula cannot masquerade as engineering savings.

4. Measure useful production and delivery units

Set the economic unit to `cost-per-successful-order`, enable the denominator quality gate, and enable delivery-cost measurement.

Create the calculation contract in `finops/unit-cost-assumptions.json`. The chapter fixture uses these explicit, illustrative assumptions:

Assumption	Northwind fixture decision	Why it must be visible
Currency	United States dollar	Mixed currencies make totals incomparable without an exchange-rate policy.
Observation window	One-hour accepted-order cohort	Numerator and denominator must describe the same population and time boundary.
Cost basis	Amortized	Commitments and discounts must not appear entirely in the purchase period.
Workload numerator	Direct worker cost plus allocated shared-runtime cost	A lower direct bill must not hide cost shifted into the shared platform.
Delivery cost	Reported separately	Pipeline economics remain visible without silently changing the workload unit.
Denominator	Quality-qualified terminal orders	Accepted, failed, duplicated, invalid, or permanently delayed work is not successful production.
Shared allocation	Documented fixed-capacity share for this fixture	Changing the allocation method is a model change, not an engineering saving.
One-time credits	Excluded and reported separately	A credit must not look like a repeatable efficiency improvement.

The fixture's baseline workload numerator is `75 + 25 = 100` dollars for 1,000 qualified orders, or `0.10` dollars per order. The candidate is `41 + 25 = 66` dollars for approximately 717.39 qualified orders, or `0.092` dollars per order. Those inputs produce the stated 34% spend reduction and 8% apparent unit-cost reduction. The values teach the calculation; they are not Northwind production prices or a universal allocation method.

A successful order is not merely accepted by the **API (Application Programming Interface)**. It must reach the valid terminal state defined by Northwind's critical outcome without duplicate charge, invalid inventory, or permanent disappearance. Report delayed and failed work separately instead of silently removing them from the cohort.

Delivery cost includes runners, artifact storage and transfer, scanners, preview environments, and repeated failed pipelines. Chapter 1 already measures queue and execution time; attach provider cost and retained evidence to the same workflow identity. Do not reduce cost by deleting the test or provenance evidence that prevents change failure.

5. Join capacity with user and queue evidence

> **Practice — Join reliability to a tested capacity envelope**
>
> Require user, queue, and recovery evidence, then connect measured requests, autoscaling, headroom, and dependency ceilings.

Enable every reliability-evidence field.

`storefront-api` can remain fast while `order-worker` falls hours behind. Chapter 6's request indicators and Chapter 9's oldest-message age therefore describe different parts of the same user journey. Recovery capacity asks a third question: after dependency or worker failure, can Northwind drain accumulated work without violating the payment provider's rate limit?

Define the observation window before evaluating the change. It must include normal peak, low traffic, scale transitions, and a recovery-shaped backlog. A seven-day average can conceal the exact hour where capacity is inadequate.

6. Build a tested capacity envelope

Enable measured requests. Add `oldest-message-age` beside processor utilization, set minimum capacity to `tested-recovery-floor`, verify maximum capacity, define failure-and-growth-tested headroom, bind scaling to the provider rate limit, and enable downscale stabilization.

Kubernetes **HPA (Horizontal Pod Autoscaler)** can evaluate multiple metrics and chooses the largest replica recommendation. Resource-based utilization also depends on resource requests being present. Horizontal Pod Autoscaling.

Processor utilization alone can stay low while workers wait on payment or the database. Oldest-message age captures delayed useful work. Conversely, scaling aggressively on backlog can overload the provider and increase failure. Bound maximum concurrency by tested downstream capacity and use per-order correctness from Chapter 9.

Minimum replicas are not sacred. They are the smallest capacity that survives routine disruption and meets measured recovery requirements. Maximum replicas are not aspirational: confirm scheduling headroom, startup time, database connections, broker partitions, payment rate limits, and cost at that scale.

7. Forecast before the invoice

> **Practice — Add predictive financial controls**
>
> Forecast expected spend, alert on forecast and anomalies, assign a service owner, and buy commitments only after measuring a stable baseline.

Enable forecasting, set budget alerts to `forecast-and-anomaly`, assign anomalies to `service-owner`, and require a baseline before commitments.

A budget threshold that fires after money is spent is historical reporting. Forecasts should explain volume, unit price, allocation coverage, planned releases, seasonal demand, and commitment assumptions. Anomaly detection finds unexpected shape; an owner decides whether it represents demand, leakage, attack, failed cleanup, price change, or allocation error.

Commitments can lower unit price while reducing flexibility. Base them on durable baseline demand, not peak autoscaling or temporary migration capacity. Keep uncertain growth and failure headroom outside the commitment unless the financial and availability trade-off is explicit.

8. Treat optimization as a production change

> **Practice — Gate, evaluate, and verify the optimization**
>
> Make rightsizing reviewed and reversible, compute the story's economics and gates, exercise recovery capacity, and verify the full contract.

Enable every optimization-change control. Set rollback capacity to `last-verified-envelope` and evidence window to `peak-and-recovery`.

Also enable expiring governance exceptions, team-scoped showback, and the prohibition on individual ranking. These controls keep temporary financial exceptions owned and stop noisy shared-cost estimates from becoming personal performance measures.

Change requests, limits, replicas, autoscaling targets, instance types, commitment coverage, or spot placement through Chapter 10's reviewed reconciliation path. Expose the change progressively where the platform supports it. Compare cost and useful outcomes against the prior envelope, and retain enough capacity to reverse before backlog or dependency pressure becomes irreversible.

Interruptible capacity is appropriate only where interruption is tested: idempotent workers with bounded leases and safe redelivery may use it, but the minimum recovery floor and stateful database need a different decision. A discount does not remove correlated interruption or replacement-delay risk.

Run the local decision engine, then the acceptance checkpoint:

```
make chapter-11-evaluate
make chapter-11-checkpoint
```

The evaluator sums only the declared workload components, keeps delivery cost separate, divides by quality-qualified terminal orders from the same cohort, and derives both percentages; the fixture does not label them. It independently evaluates the success, completion, and oldest-message-age gates. The same run restores the verified processing profile and simulates 30 minutes of backlog recovery, proving the queue drains without crossing the provider limit.

This is a deterministic decision-system and capacity-recovery simulation, not a live bill, cluster resize, or autoscaler. The checkpoint requires its calculated results in addition to the policy contract. Production must also validate billing-export delay, missing allocation, discounts and credits, currency and amortization rules, telemetry gaps, autoscaler failure, node shortage, provider throttling, and cost during incident recovery.

9. Reject a cheap but slow worker fleet

Severity: delayed asynchronous order completion and lost recovery margin after aggressive rightsizing.
Potential blast radius: orders entering the worker backlog during the optimization window.
Bounded by: progressive capacity change, oldest-age and completion gates, dependency-aware maximums, and a retained last verified envelope.
Primary principles: trustworthy evidence, blast-radius control, explicit contracts, reconciliation, and recovery.

> **Practice — Diagnose false savings**
>
> Evaluate the lower-cost worker fleet and prove that spend reduction cannot approve degraded backlog and recovery behavior.

```
make chapter-11-break
```

The report must show `spend_reduction_is_real`, `unit_cost_does_not_prove_health`, `backlog_harm_is_detected`, `optimization_is_rejected`, and `recovery_restores_verified_capacity` as true. Spend reduction is accepted as a fact; retention of the change is rejected because the computed 780-second completion and 1,900-second oldest age exceed independent expectations. Recovery passes only when the simulated backlog reaches zero within the provider ceiling.

Restoring replicas is an action, not recovery. Reconcile the last verified envelope, cap the recovery ramp below provider and database limits, verify oldest-message age falls continuously, confirm order completion and success recover, reconcile payment and inventory outcomes, and measure the temporary recovery cost. Do not declare success while the backlog is merely smaller.

10. Production reality

Best Practice: allocate spend, measure cost per useful outcome, join cost with reliability, derive capacity from workload evidence, forecast before thresholds, and treat optimization as a reversible production change.

Production Practice: qualify billing data delay, allocation gaps, shared-cost formulas, discounts, commitments, telemetry failure, seasonality, dependency ceilings, recovery demand, interruptible capacity, and organizational ownership. Cost targets are constraints for teams and systems; do not convert noisy infrastructure allocation into individual-engineer rankings.

11. What changed

Before	After
Total spend lacked operational ownership.	**Direct and shared cost have explicit service and organizational dimensions.**
Cheap successful requests defined efficiency.	**Cost is normalized by correct terminal orders and constrained by delay and recovery.**
Currency, window, allocation, discounts, and denominator were implicit.	**A reviewed assumption contract makes the unit reproducible and exposes model changes.**
Processor utilization controlled worker scale.	**Work age, requests, headroom, and dependency capacity define an envelope.**
Budgets reported overspend after occurrence.	**Forecast and anomalies create owned, earlier decisions.**
Rightsizing changed production immediately.	**Optimization is reviewed, progressive, evidence-gated, and reversible.**
Cost reports could rank individuals.	**Team showback informs system decisions and exceptions expire.**

Durable outputs

Artifact	Location	Keep it because
Unit-cost assumption contract	finops/unit-cost-assumptions.json	It makes currency, cohort, numerator, denominator, allocation, and accounting treatments reviewable.
Cost-and-capacity decision evidence	evidence/chapter-11-green.json	It retains the calculated economics, reliability decision, and verified recovery result together.

What You Learned

Cost efficiency is a production outcome, not a lower invoice in isolation. You can now state and reproduce a unit-cost model, allocate spend, define a quality-gated economic unit, join economics to synchronous and asynchronous reliability, construct a tested capacity envelope, forecast anomalies, and reject savings that remove recovery margin.

Prove It

> **Independent Practice — Place interruptible worker capacity safely**
>
> Move 60% of `order-worker` capacity to instances that may disappear with two minutes' notice during Northwind's busiest hour.

Define the non-interruptible floor, termination and lease behavior, per-order idempotency dependency, autoscaling signals, replacement-time assumption, provider limit, backlog and completion gates, cost baseline, interruption experiment, rollback trigger, recovery ramp, and evidence needed before expanding beyond 60%. Explain how correlated loss differs from ordinary Pod disruption.

Next

Northwind can now optimize cost without silently spending reliability or recovery margin. Chapter 12 coordinates diagnosis, rollback or roll-forward, communication, and verified recovery when a production change still fails.

12 — Recover a Failed Production Change

> **Outcome:** Coordinate a production incident, choose a safe rollback or roll-forward from evidence, and prove recovery rather than merely completing a mitigation.

Current Northwind state: Chapters 1–11 prevent, bound, and expose many unsafe changes, but no control eliminates change failure. Northwind still needs an operating model for coordinated response.
Prerequisites: Chapters 1–11, especially artifact identity, observability, progressive delivery, data compatibility, asynchronous processing, and GitOps.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. A healthy endpoint and 600 stuck orders

A reviewed `storefront-api` candidate begins publishing queue schema `v2`. The stable `order-worker` understands only `v1`. Order acceptance remains at 99.8%, yet decode errors reach 18%, the oldest message is 20 minutes old, and 600 `v2` messages cannot reach a terminal state.

Rolling the producer back stops new `v2` messages, but it cannot make the stable consumer understand messages already in the queue. Purging the queue would make the dashboard greener by discarding customer work.

> How should Northwind coordinate a mixed recovery when rollback alone is unsafe?
>
> **Practice — Establish the uncoordinated baseline**
>
> Check out the start state and prove that evidence exists but incident authority, communication, mitigation, and recovery contracts are red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-12 chapter-12-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-12-baseline
```

Notice that the baseline already diagnoses a schema mismatch. Detection is not coordination, and a plausible hypothesis is not recovery.

2. The production model: incident command as a control system

> *Theory — Coordinated mitigation and verified recovery*
>
> Decide who controls the response, how evidence becomes action, and what must remain true before declaring recovery.

An incident response loop has five transitions:

```
declare → establish control → test hypotheses → mitigate → verify recovery
                └──────── communicate throughout ────────┘
```

Declaration makes urgency and impact explicit. Named roles prevent one overloaded responder from operating, coordinating, and communicating simultaneously. A change freeze reduces interference while a shared incident record preserves decisions, evidence, and rejected hypotheses.

Mitigation reduces current harm. Recovery proves the critical outcome is healthy and durable. Rollback, failover, scaling, or queue draining are actions; none proves recovery by itself.

Google's incident guidance separates command, operations, and communications, recommends early declaration and regular updates, and prioritizes mitigation before root-cause analysis. It also treats incident training and blameless learning as capabilities that must be practiced. Incident Response and Managing Incidents.

3. Declare impact and separate authority

> **Practice — Establish incident control**
>
> Define impact-based declaration, distinct response roles, a change freeze, an evidence record, and bounded emergency access.

Edit `incident/response-contract.json`. Set declaration criteria to `user-impact` and use **SEV-1 (Severity 1)** as Northwind's local classification.

Assign four distinct owners:

- incident commander: sets priorities, resolves contention, and owns the response rhythm;
- operations lead: executes and verifies technical changes;
- communications lead: sends audience-appropriate updates at the declared cadence;
- scribe: preserves the timeline, hypotheses, decisions, and evidence.

Enable `change_freeze`, `single_source_of_truth`, and `hypothesis_log`. Set break-glass access to `individual-time-bound`. The incident commander should not become a shared administrator; emergency actions still need attributable identity, expiry, and later review.

These roles are responsibilities, not guaranteed headcount. A small incident may combine roles, but Northwind's highest severity requires distinct owners because the coordination load and blast radius justify them.

4. Diagnose before selecting rollback

> **Practice — Bind mitigation to the queue evidence**
>
> Reject a full rollback, preserve accepted work, and define a mixed recovery using verified and reviewed artifacts.

Read `fixtures/incident/incompatible-queue-release.json`. The **API (Application Programming Interface)** success ratio is healthy, but three facts support the schema-mismatch hypothesis:

- the candidate producer emits `v2`;
- the stable consumer accepts only `v1`;
- decode failures and pending `v2` messages appeared with the release.

Healthy acceptance therefore does not prove completed orders. Chapter 6's request evidence and Chapter 9's queue evidence describe different stages of the same user journey.

In `incident/response-contract.json`, set mitigation to `rollback-producer-rollforward-consumer`. Preserve the queue, require the producer rollback to use Chapter 2's verified stable digest, and require the compatible consumer roll-forward to pass the reviewed delivery path.

This is not a preference for roll-forward. It follows from state already produced: rollback can stop new incompatible messages, but only a `v1`/`v2` consumer can finish the accepted `v2` work. Do not mutate or purge evidence to make reconciliation appear successful.

5. Communicate while recovery runs

> **Practice — Execute the response trace**
>
> Add an owned communication cadence and run the mixed mitigation against independent recovery expectations.

Set a 15-minute cadence, enable internal and external audiences, and require every update to state the next update time. Communicate known impact, current mitigation, uncertainty, and the next decision point; do not wait for a complete root cause.

In the recovery contract, require at least two consecutive passing samples and all five outcomes:

- successful order completion;
- bounded oldest-message age;
- every accepted order reaches a terminal state;
- no duplicate charge;
- reviewed desired state and actual state agree.

Run the exercise:

```
make chapter-12-exercise
```

The trace must declare the incident, separate roles, freeze unrelated change, support the schema hypothesis, reject full rollback, execute the mixed mitigation, emit three status updates, and evaluate recovery samples. Each update carries the observed impact, current mitigation state, remaining uncertainty, and an explicit `next_update_minute`; the checkpoint validates the generated content and cadence rather than trusting the communication flags. Thresholds come from `incident/recovery-expectations.json`, not from the response contract that is being evaluated.

This is a deterministic incident exercise. It does not page responders, route production traffic, deploy a real consumer, or update a public status system. Production exercises must validate those integrations, telemetry freshness, access availability, control-plane behavior, stakeholder reachability, and decisions under incomplete evidence.

6. Prove that mitigation is not recovery

Severity: accepted orders cannot complete after an incompatible asynchronous release.
Potential blast radius: every order published as `v2` while only `v1` consumers are available.
Bounded by: progressive producer exposure, preserved queue state, verified rollback identity, reviewed compatible consumer, idempotent processing, and explicit recovery gates.
Primary principles: trustworthy evidence, blast-radius control, explicit contracts, reconciliation, and recovery.

> **Practice — Diagnose and recover the incompatible release**
>
> Prove that one green observation is insufficient and that all pending work reaches a correct terminal state without purging the queue.

```
make chapter-12-break
```

The first recovery sample still fails because the oldest message is 800 seconds old. The following samples pass, but recovery is declared only after two consecutive passes and zero pending messages. The trace must also show zero duplicate charges.

The operational sequence is:

1. stop new `v2` production by reconciling the verified stable producer digest;
2. deploy the reviewed dual-schema consumer;
3. retain and drain the queue under Chapter 9's idempotency controls;
4. observe completion, age, duplicates, and desired-versus-actual identity;
5. maintain the communication cadence until recovery criteria remain satisfied.

If a second symptom appears, record and test a new hypothesis. Do not rewrite the timeline around the eventual explanation.

7. Preserve learning and verify the contract

> **Practice — Make incident learning actionable**
> Preserve a blameless timeline and require every follow-up action to have an owner, deadline, and verification method.

Enable the four learning controls in `incident/response-contract.json`: blameless review, preserved timeline, contributing conditions, and verifiable actions.

Blameless does not mean consequence-free or vague. It means examining how review, compatibility contracts, rollout ordering, queue visibility, and authority made the action reasonable or the failure possible. “Engineer should be more careful” is not a control. A consumer compatibility test with an owner, delivery date, and failing fixture is verifiable.

Run the complete checkpoint:

```
make chapter-12-checkpoint
```

The checkpoint combines declared policy with executed trace evidence. Green fields alone cannot pass it: the incident must actually be declared, the mismatch diagnosed, the unsafe rollback rejected, communications emitted, and recovery sustained.

8. Production reality

Best Practice: declare early from user impact, separate command from operations and communications, freeze interference, keep one evidence record, mitigate before pursuing full causality, communicate on a cadence, and verify recovery from user and system outcomes.

Production Practice: adapt severity, roles, legal and regulatory notification, customer communication, escalation, break-glass access, evidence retention, and recovery windows to the organization. Exercise missing responders, stale telemetry, inaccessible runbooks, failed identity providers, unavailable GitOps controllers, and mitigations that create new state.

9. What changed

Before	After
A responder decided informally whether failure was an incident.	**User impact triggers a defined severity and response.**
One person coordinated, operated, and communicated.	**Distinct roles own command, operations, communications, and evidence.**
Healthy acceptance obscured stuck asynchronous work.	**Request, schema, queue, and terminal-outcome evidence test one hypothesis.**
Rollback was the default response.	**Existing production state determines rollback, roll-forward, or a mixed recovery.**
Completing a deployment implied recovery.	**Sustained business, queue, duplication, and reconciliation evidence proves it.**
Learning produced generic recommendations.	**A blameless timeline creates owned, dated, verifiable controls.**

Durable outputs

Artifact	Location	Keep it because
Incident response and recovery contract	incident/response-contract.json	It records declaration, role authority, change freeze, hypothesis handling, mitigation, communication, recovery, and learning requirements.
Executed incident evidence	evidence/chapter-12-green.json	It retains the evaluated trace, communications, mitigation decision, and sustained recovery result instead of preserving only a completed checklist.

What You Learned

Incident response is a production control system, not an improvised sequence of commands. You can now declare from impact, separate authority, preserve evidence, test hypotheses, choose mitigation from existing state, communicate during uncertainty, distinguish action from recovery, and convert learning into verifiable engineering work.

Prove It

> **Independent Practice — Recover with a missing control plane**
>
> Redesign the response when the GitOps controller is unavailable while incompatible `v2` messages continue accumulating.

Define declaration and escalation, role ownership, how the change freeze is enforced, independently attributable break-glass access, the verified producer artifact, the reviewed compatible consumer, an auditable temporary apply path, queue preservation, communications, recovery gates, controller restoration, reconciliation back to reviewed intent, evidence retention, and the conditions for removing emergency access. Explain which actions are mitigation and which observations prove recovery.

Next

Northwind can recover from a failed change while its durable systems remain available. Chapter 13 handles the harder case: reconstructing production service and data from durable evidence after state or control-plane loss.

13 — Restore Production from Durable Evidence

> **Outcome:** Reconstruct Northwind after state and control-plane loss, measure recovery objectives from the executed timeline, and reconcile every accepted order before releasing traffic.

Current Northwind state: Chapter 12 coordinates recovery while production state remains available. Northwind has not proved that its backups, identity, infrastructure declarations, queue evidence, and reviewed intent can reconstruct a lost environment.
Prerequisites: Chapters 1–12, PostgreSQL backup fundamentals, infrastructure as code, workload identity, asynchronous processing, and GitOps.
Implementation: `books/labs/devops/northwind/`
Guided time: approximately 90–120 minutes.

1. A successful backup job and no production

Northwind loses its production database and cluster control plane. The newest PostgreSQL backup is ten minutes old and its scheduled job reported success. During restore, its content digest does not match the manifest. An older base backup is valid, archived database changes reach five minutes before the incident, and five accepted orders exist only in the durable queue and payment provider.

Restoring the newest object would be fast and wrong. Restoring only the older database would produce a consistent database while losing accepted business work. Recreating workloads before identity, data, and queue dependencies would turn disaster recovery into an uncontrolled second incident.

> Can Northwind reconstruct production from independently verified evidence and prove every accepted order is correct within its recovery objectives?
>
> **Practice — Establish the unproved restore baseline**
>
> Check out the start state and prove that useful backup evidence exists while objectives, authority, restore ordering, and recovery validation remain red.

```
cd books/labs/devops/northwind
git switch -c my-chapter-13 chapter-13-start
python3.12 -m venv .venv
source .venv/bin/activate
make bootstrap
make chapter-13-baseline
```

Several evidence checks are already green: the fixture contains a usable older backup, a continuous log chain, and reconcilable business records. That does not prove Northwind has the authority, procedure, or gates to restore them safely.

2. The production model: reconstruct, reconcile, release

> *Theory — Recovery objectives measured by restoration*
>
> Decide which evidence can reconstruct the system, how much state and time may be lost, and what must reconcile before traffic returns.

RPO (Recovery Point Objective) bounds the acceptable gap between the incident and the latest recoverable state. **RTO (Recovery Time Objective)** bounds the elapsed time to restore the required service outcome. Neither is established by writing a target in a document:

```
durable evidence → isolated reconstruction → dependency restore
       → business reconciliation → traffic release

measured RPO = incident time − latest verified recovery point
measured RTO = verified recovery time − incident declaration time
```

A backup is an input. Recovery is an observed outcome from a clean restore, integrity validation, log replay, application startup, and business reconciliation. The restore must not destroy its source evidence, and the production identity must not be the only identity capable of reading recovery material.

PostgreSQL continuous archiving combines a base backup with **WAL (Write-Ahead Log)** files and supports **PITR (Point-in-Time Recovery)**. Successful recovery requires an unbroken WAL sequence from the base backup to the selected recovery point; PostgreSQL also recommends inspecting the restored database before normal users connect. PostgreSQL continuous archiving and PITR.

For Kubernetes control-plane loss, official guidance identifies regular etcd snapshots as necessary recovery material. Application resources may instead be reconstructed from reviewed Git intent, but that does not replace database, queue, provider, identity, or cluster-bootstrap evidence. Operating etcd clusters for Kubernetes.

3. Make recovery evidence survive production

> **Practice — Define independently recoverable evidence**
>
> Bound recovery objectives and protect backups from the failure or identity compromise that destroyed production.

Edit `recovery/restore-contract.json`. Set the declared RPO to 10 minutes and RTO to 60 minutes, then require both to be measured by the exercise.
Enable encrypted, immutable backup storage, an isolated recovery identity, manifest verification before restore, periodic restore testing, and continuous WAL coverage. Immutability needs a retention and deletion model; otherwise the same production credential or attacker can erase both primary state and recovery evidence. Encryption needs recoverable keys whose ownership and rotation are tested independently of the lost environment.
Set emergency access to `individual-time-bound`, make source evidence read-only during recovery, and audit restore actions. A shared emergency credential provides access but destroys attribution. Copy verified evidence into the recovery environment; do not rewrite the only backup, WAL archive, or Git history to fit the desired result.

4. Select evidence before rebuilding

> **Practice — Reject corruption and select a recovery point**
>
> Verify candidate backups by digest, reject the newest corrupted object, and bind an older base backup to a continuous WAL chain.

Inspect:

```
fixtures/recovery/backup-manifest.json
fixtures/recovery/backups/base-001.json
fixtures/recovery/backups/base-002.json
fixtures/recovery/wal.json
```

The manifest considers the newest candidate first. Its computed digest does not match, so it must remain evidence but cannot become restore input. `base-001` matches its manifest and its WAL chain continues from minute −60 to the last safe database point at minute −5.

The resulting measured RPO is five minutes, not the age of the base backup. The base image supplies a consistent starting point; WAL replay advances it. A missing segment would invalidate the later recovery point even if every other segment and the base backup were healthy.

5. Restore dependencies without creating a second writer

> **Practice — Reconstruct in dependency order**
>
> Restore identity, infrastructure, state, workloads, and reconciliation authority while traffic remains blocked.

Set `clean_environment` and declare this order:

1. `emergency-identity`
2. `infrastructure`
3. `postgres-and-wal`
4. `durable-queue`
5. `workloads`
6. `gitops-controller`

Require the GitOps controller to return last and keep traffic blocked until validation. Emergency identity first makes every later action attributable. Infrastructure creates isolated network, compute, storage, and key dependencies. PostgreSQL and WAL establish the database recovery point. The durable queue returns accepted work that may be newer than that point. Workloads start without public traffic and reconcile state. Only then should the controller continuously enforce reviewed intent.

Restoring the controller early can race the recovery operator, prune temporary resources, or start workloads against incomplete dependencies. After bootstrap, reconcile any emergency change back into reviewed intent and revoke break-glass access.

6. Reconcile business state and measure recovery

> **Practice — Execute and verify the restore**
>
> Run the restore trace, reconcile durable sources, and release traffic only when measured objectives and business invariants pass.

Enable all five reconciliation gates: database/queue/provider comparison, terminal orders, duplicate charges, inventory, and desired-versus-actual state.
Run:

```
make chapter-13-restore
make chapter-13-checkpoint
make chapter-13-break
```

The exercise computes file digests, rejects `base-002`, selects `base-001`, verifies WAL continuity, and executes the dependency timeline. Database recovery produces 995 orders; the durable queue contributes the five accepted after the database recovery point. Provider evidence must show 1,000 payments, with zero duplicate charges and zero inventory discrepancies. Reviewed and restored revisions must match.
The final event measures an RPO of five minutes and derives an RTO of 52 minutes by accumulating the fixture's component restore durations and business-reconciliation time. It compares both results with independent expectations in `recovery/objectives.json`. Traffic release is an output of all gates, not a configured success flag. The `chapter-13-break` target applies additional assertions to this same restore trace: it does not introduce a second failure.
This is a deterministic restore decision and reconciliation simulation. The fixture base images are not real PostgreSQL data directories, the WAL fixture is not binary WAL, and no cluster is created. Production exercises must execute the actual backup client, key recovery, network isolation, PostgreSQL replay, queue restore, provider reconciliation, cluster bootstrap, GitOps recovery, **DNS (Domain Name System)** or routing transition, load validation, and observability path.
The restore trace is the chapter's failure exercise:

Severity: production data and control-plane loss with a corrupted newest backup.
Potential blast radius: all Northwind order processing and every state transition after the selected recovery point.
Bounded by: isolated immutable evidence, digest verification, continuous WAL, clean reconstruction, blocked traffic, idempotent reconciliation, and independent recovery objectives.
Primary principles: trustworthy evidence, explicit contracts, reconciliation, blast-radius control, and recovery.

The report rejects the newest candidate, selects the older verified backup, replays through minute −5, and reconciles 995 database orders plus five durable queue orders into 1,000 correct terminal orders. `base-002` has a recorded **SHA-256 (Secure Hash Algorithm 256-bit)** digest for its pre-tamper bytes; its current bytes differ. Do not repair the object or rewrite its expected digest. Preserve both values for diagnosis and retention-policy review.

Restoring a database is an action. Starting Pods is an action. Releasing traffic is safe only after the critical Northwind outcome, external payment evidence, inventory, duplication, desired state, and measured RPO/RTO all pass.

7. Production reality

Best Practice: define recovery objectives from business tolerance, isolate and protect recovery evidence, verify integrity before use, restore into a clean environment, respect dependencies, block traffic, reconcile independent durable sources, and measure objectives through regular exercises.

Production Practice: validate database size and replay rate, backup lag, retention and legal holds, key and identity recovery, region or account isolation, infrastructure-state recovery, DNS and certificate dependencies, queue semantics, external-provider evidence, controller bootstrap, observability availability, staffing, and exercise frequency. Different components may require different objectives and recovery mechanisms.

8. What changed

Before	After
Backup-job success implied recoverability.	**Integrity and clean restoration establish usable evidence.**
RPO and RTO were declared aspirations.	**The executed recovery point and timeline measure them.**
Production identity controlled recovery material.	**Isolated, attributable authority survives production compromise.**
The newest backup was selected by age.	**Integrity and a continuous WAL chain select the recovery point.**
Components could start in arbitrary order.	**Dependencies restore while public traffic remains blocked.**
Database availability implied recovery.	**Database, queue, provider, inventory, orders, and reviewed intent reconcile.**

Durable outputs

Artifact	Location	Keep it because
Restore contract and independent objectives	recovery/restore-contract.json and recovery/objectives.json	They separate recovery procedure from the recovery-point and recovery-time limits used to judge it.
Executed restoration evidence	evidence/chapter-13-green.json	It preserves backup selection, integrity, replay, reconstruction order, business reconciliation, measured objectives, and the traffic-release decision.

What You Learned

Disaster recovery is evidence-driven reconstruction, not backup creation. You can now define measurable recovery objectives, protect evidence independently, reject a corrupted recent backup, select a valid base and log chain, restore control dependencies safely, reconcile cross-system business state, and withhold traffic until recovery is proved.

Prove It

> **Independent Practice — Recover when the WAL archive has a gap**
>
> Redesign the exercise when the newest valid base backup is 45 minutes old and the WAL segment covering minutes −18 to −12 is missing.

Determine the latest defensible recovery point, whether the 10-minute RPO can still be claimed, how queue and payment evidence may reconstruct or compensate for missing database state, which orders require human review, how inventory is protected, what traffic may return, who approves an objective breach, what evidence must be preserved, and which verified change prevents recurrence. Do not invent continuity that the archive cannot prove.

Next

Northwind can now reconstruct this production environment from durable evidence within the chapter's tested scenario. Defining restore service-level objectives across a service portfolio, designing for regional loss, governing backup programs, and running recurring recovery game days belong to the **SRE (Site Reliability Engineering)** book rather than this DevOps delivery path.

Conclusion — An Evidence-Driven Delivery Path

Northwind began with a familiar failure: a cheap source defect took more than twenty minutes to reach an engineer because expensive work ran first. That problem looked like pipeline tuning. Solving it exposed the larger production system behind every change.

A fast pipeline was useful only when its evidence was trustworthy. A trusted pipeline still could not prove that production received the bytes it evaluated. A verifiable artifact needed reconciled infrastructure, a bounded runtime, attributable dependency access, and telemetry capable of explaining user-visible behavior. Those signals then became the control input for progressive release.

The same reasoning continued beyond deployment. A safe binary release could still break shared data. A compatible database change could still produce duplicate external effects under message redelivery. Reviewed intent could still be operationally harmful. A lower bill could still mean fewer correct outcomes. A rollback could still leave incompatible state behind. A completed backup job could still produce unusable recovery material.

The book's central argument is therefore broader than tool automation:

> Production DevOps is the design of evidence, authority, state transitions, and recovery across the complete delivery path.

What Northwind can now do

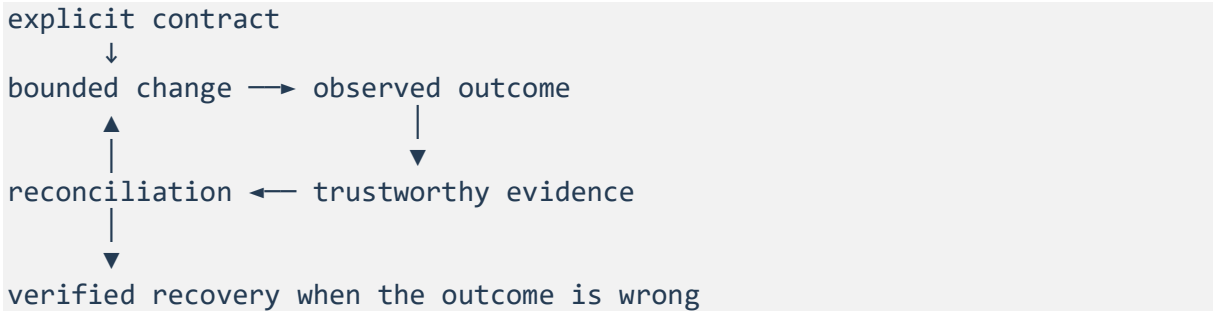
Northwind can now:

- reject inexpensive source failures early without giving pull-request code publication authority;
- build one candidate, identify it immutably, bind evidence to it, and promote it without rebuilding;
- adopt and reconcile existing infrastructure through reviewed plans and protected apply authority;
- define scheduling, health, disruption, network, and execution contracts for Kubernetes workloads;
- replace ambient reusable credentials with attributable, scoped, short-lived workload access;
- correlate production behavior across logs, metrics, traces, dependencies, releases, and user outcomes;
- expose a bounded candidate cohort and advance, pause, or abort from trustworthy evidence;
- evolve persistent data while stable and candidate versions coexist;
- make at-least-once processing converge on one correct business effect;
- reconcile reviewed intent through a bounded pull controller without allowing it to approve itself;
- evaluate workload cost through useful outcomes, reliability gates, and tested recovery capacity;
- coordinate a failed-change response with explicit roles, evidence, communication, and sustained recovery gates;
- reconstruct this Northwind environment from verified durable evidence and reconcile cross-system business state before traffic returns.

These capabilities form one system. They are not thirteen independent checklists.

The five principles as one control loop

The recurring principles now connect end to end:



Explicit contracts make identity, authority, intent, compatibility, outcomes, and failure behavior reviewable.

Blast-radius control limits how much production is exposed while evidence remains incomplete.

Trustworthy evidence separates observation from expectation and prevents a mechanism from approving itself.

Reconciliation turns disagreement among desired, recorded, external, and actual state into an owned transition.

Recovery requires proof that the critical outcome is healthy again—not merely that an operator completed an action.

What this book does not claim

The companion lab is intentionally deterministic. It proves decision logic, contracts, failure interpretation, and recovery gates without requiring a live cloud estate. It does not prove that a particular organization's runners, registry, identity provider, Kubernetes control plane, telemetry pipeline, payment provider, billing export, incident system, or backup platform behaves as the fixture does.

Production adoption requires replacing each simulated boundary with observed evidence from the real implementation. Values such as resource requests, latency thresholds, rollout cohorts, recovery objectives, and capacity ceilings must be measured under the actual workload and failure modes.

The scope is also deliberately bounded:

- The DevSecOps book extends artifact and identity foundations into supply-chain policy, vulnerability management, secret governance, detection, and compromise response.
- The Platform Engineering book turns repeated delivery capabilities into owned products, paved roads, tenant boundaries, and fleet lifecycle.
- The **SRE (Site Reliability Engineering)** book owns service-level objective programs, error-budget governance, on-call systems, regional-loss architecture, recurring game days, and reliability learning across a service portfolio.

Chapter 13 proves that Northwind can reconstruct this environment in one tested scenario. It does not claim that Northwind has completed an enterprise disaster-recovery program.

The production question to keep asking

When evaluating a new tool, control, or practice, ask:

> What production decision does this enable, what authority does it require, what evidence can falsify it, how is its blast radius bounded, and how will we prove recovery when it fails?

That question keeps concepts subordinate to production work. It also prevents “best practice” from becoming configuration copied without workload evidence.

Northwind's implementation is complete for the promise of this book. The lasting skill is not reproducing its configuration, workflow, or manifest values. It is being able to redesign the same delivery path when the organization, workload, dependencies, failure modes, and constraints are different.

Glossary and Abbreviations

Abbreviations

Abbreviation and full form	Use in this book
AI (Artificial Intelligence)	A drafting aid whose output remains untrusted source until reviewed and verified by the delivery controls.
API (Application Programming Interface)	A service or provider boundary invoked by software.
CD (Continuous Delivery)	Keeping a change deployable through an automated, controlled path.
CI (Continuous Integration)	Producing fast, trustworthy evidence about proposed source changes.
CIDR (Classless Inter-Domain Routing)	Expressing an IP address range used by network configuration.
DAG (Directed Acyclic Graph)	Modeling pipeline jobs and dependencies to calculate the critical path.
DNS (Domain Name System)	Resolving service names and participating in recovery traffic transitions.
HPA (Horizontal Pod Autoscaler)	Adjusting Kubernetes replicas from observed metrics within declared bounds.
HTTP (Hypertext Transfer Protocol)	Carrying requests and trace context across service boundaries.
IAM (Identity and Access Management)	Defining which subjects can perform which actions.
OIDC (OpenID Connect)	Providing verifiable identity claims used by hosted builds or workload federation.
PDB (PodDisruptionBudget)	Constraining voluntary Kubernetes Pod evictions.
PITR (Point-in-Time Recovery)	Restoring a database to a selected point by replaying archived changes.
RPO (Recovery Point Objective)	Maximum acceptable gap between an incident and the latest recoverable state.
RTO (Recovery Time Objective)	Maximum acceptable elapsed time to restore the required outcome.

Abbreviation and full form	Use in this book
S3 (Simple Storage Service)	**The Amazon object-storage service used in the Terraform backend example.**
SBOM (Software Bill of Materials)	**A structured inventory of components associated with an artifact.**
SEV-1 (Severity 1)	**Northwind's local classification for its highest-severity incident exercise.**
SHA-256 (Secure Hash Algorithm 256-bit)	**The digest algorithm used to detect changed fixture bytes.**
SLI (Service-Level Indicator)	**A measurement of a user-visible service outcome.**
SLO (Service-Level Objective)	**The acceptable target for an SLI over a defined window.**
SLSA (Supply-chain Levels for Software Artifacts)	**A framework for reasoning about build provenance and platform assurance.**
SPDX (Software Package Data Exchange)	**The SBOM document format used by the artifact exercise.**
SQS (Simple Queue Service)	**The Amazon queue service cited for at-least-once delivery behavior.**
SRE (Site Reliability Engineering)	**Reliability engineering through service objectives, operations, and learning.**
WAL (Write-Ahead Log)	**PostgreSQL change records replayed after a base backup.**
W3C (World Wide Web Consortium)	**The standards organization responsible for Trace Context.**

Production terms

Actual state
The observed state of a running system or external dependency, which may disagree with declared or recorded state.

Artifact digest
A content-derived immutable identifier. A tag may move; a digest identifies specific bytes.

Attestation
Authenticated evidence containing claims about an artifact or process. A valid signature authenticates the signer but does not automatically make the signer trusted for the claimed action.

Blast radius
The users, workloads, data, environments, or authority exposed to a change or failure.

Break-glass access
Exceptional emergency authority that should be attributable, time-bound, audited, and removed after reconciliation.

Burn rate
The rate at which current failures consume the unreliability permitted by a service-level objective.

Canary
A bounded production cohort exposed to a candidate release before wider progression.

Change failure
A production change that degrades an intended outcome and requires mitigation, rollback, roll-forward, or another corrective transition.

Critical path
The longest dependent sequence in a pipeline; it determines total execution time when independent work can run concurrently.

Desired state
Reviewed intent describing what the system should become.

Drift
Disagreement between declared, recorded, and observed infrastructure or workload state.

Error budget
The amount of unreliability permitted by a service-level objective over its window.

Idempotency
The property that repeated attempts for the same business intent converge on one intended effect.

Inbox pattern
A durable consumer-side record that identifies processed operations and participates in the business transaction.

Immutable artifact
A release subject promoted by content identity rather than rebuilt or selected through a mutable name.

Mitigation
An action that reduces current harm. Mitigation is not equivalent to verified recovery.

Outbox pattern
A durable event-publication intent committed in the same transaction as local business state and relayed afterward.

Provenance
Evidence binding an artifact subject to build facts such as source, revision, builder, and build type.

Reconciliation
Observing disagreement and applying or proposing a controlled transition toward selected intent.

Recovery
The verified restoration of the critical production outcome, not merely completion of a corrective action.

Recovery capacity
Capacity retained or obtainable to process accumulated work after disruption without overloading dependencies.

Red capability
A runnable checkpoint result proving that a production capability is absent or unsafe.

Stable artifact
The immutable release identity currently trusted to serve normal production traffic.

Terminal state
A business state in which accepted work has completed successfully or reached an explicitly handled final failure without disappearing.

Trust expectation
Independent policy describing which subject, source, builder, issuer, audience, or outcome is acceptable.

Unit economics
Cost normalized by a useful, quality-gated production outcome rather than by raw requests or infrastructure quantity alone.

Workload identity
An attributable runtime subject exchanged for short-lived, audience-bound dependency access without distributing a reusable credential where federation is supported.

References

The chapter text cites sources at the decision they support. This consolidated list is organized by chapter for review and maintenance. Product behavior and documentation change; verify version-specific details before applying an example to production.

Chapter 2 — Verifiable artifacts

- **SLSA (Supply-chain Levels for Software Artifacts)** build requirements
- SLSA artifact verification guidance
- Docker build attestations
- GitHub artifact attestations

Chapter 3 — Infrastructure reconciliation

- Terraform state
- Terraform remote state
- Terraform state locking
- Terraform import
- Terraform **S3 (Simple Storage Service)** backend
- Terraform plan
- Create and apply a saved Terraform plan
- Manage Terraform resource drift
- AWS Certificate Manager managed renewal
- AWS Certificate Manager EventBridge events

Chapter 4 — Kubernetes runtime

- Kubernetes resource management
- Kubernetes probes
- Kubernetes disruptions
- Kubernetes networking

Chapter 5 — Workload identity

- Kubernetes service accounts
- Kubernetes projected volumes
- Amazon **IAM (Identity and Access Management)** roles for service accounts

Chapter 6 — Observability

- OpenTelemetry signals
- OpenTelemetry instrumentation

- OpenTelemetry context propagation
- OpenTelemetry deployment attributes

Chapter 7 — Progressive delivery

- Google **SRE (Site Reliability Engineering)** Workbook: Canarying Releases
- Kubernetes Deployments

Chapter 8 — Data compatibility

- PostgreSQL ALTER TABLE

Chapter 9 — Asynchronous work

- Amazon **SQS (Simple Queue Service)** at-least-once delivery
- Stripe idempotent requests

Chapter 10 — GitOps

- OpenGitOps principles
- Argo **CD (Continuous Delivery)** sync waves

Chapter 11 — Cost and capacity

- FinOps Framework
- Kubernetes Horizontal Pod Autoscaling

Chapter 12 — Incident response

- Google SRE Workbook: Incident Response
- Google SRE Book: Managing Incidents

Chapter 13 — Durable reconstruction

- PostgreSQL continuous archiving and point-in-time recovery
- Kubernetes: Operating etcd clusters